```
!pip install -q transformers datasets accelerate transformer-lens
```

```
  Preparing metadata (setup.py) ... done
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 977.7/977.7 kB 23.3 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 10.8/10.8 MB 44.5 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 56.4/56.4 kB 2.4 MB/s eta 0:00:00
  Building wheel for transformers-stream-generator (setup.py) ... done
```

```
import torch
from transformers import AutoTokenizer, AutoModelForCausalLM

model_name = "EleutherAI/pythia-160m"

tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForCausalLM.from_pretrained(
    model_name,
    torch_dtype=torch.float16,
    device_map="auto"
)

model.eval()
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:112: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and fast
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN t
config.json: 100%  569/569 [00:00<00:00, 13.3kB/s]
tokenizer_config.json: 100%  396/396 [00:00<00:00, 15.8kB/s]
tokenizer.json: 100%  2.11M/2.11M [00:00<00:00, 24.2MB/s]
special_tokens_map.json: 100%  99.0/99.0 [00:00<00:00, 2.50kB/s]
[transformers] `torch_dtype` is deprecated! Use `dtype` instead!
model.safetensors: 100%  375M/375M [00:02<00:00, 138MB/s]
Loading weights: 100%  148/148 [00:00<00:00, 612.27it/s]
GPTNeoXForCausalLM(
  (gpt_neox): GPTNeoXModel(
    (embed_in): Embedding(50304, 768)
    (emb_dropout): Dropout(p=0.0, inplace=False)
    (layers): ModuleList(
      (0-11): 12 x GPTNeoXLayer(
        (input_layernorm): LayerNorm((768,), eps=1e-05, elementwise_affine=True)
        (post_attention_layernorm): LayerNorm((768,), eps=1e-05, elementwise_affine=True)
        (post_attention_dropout): Dropout(p=0.0, inplace=False)
        (post_mlp_dropout): Dropout(p=0.0, inplace=False)
        (attention): GPTNeoXAttention(
          (query_key_value): Linear(in_features=768, out_features=2304, bias=True)
          (dense): Linear(in_features=768, out_features=768, bias=True)
        )
        (mlp): GPTNeoXMLP(
          (dense_h_to_4h): Linear(in_features=768, out_features=3072, bias=True)
          (dense_4h_to_h): Linear(in_features=3072, out_features=768, bias=True)
          (act): GELUActivation()
        )
      )
    )
    (final_layer_norm): LayerNorm((768,), eps=1e-05, elementwise_affine=True)
    (rotary_emb): GPTNeoXRotaryEmbedding()
  )
  (embed_out): Linear(in_features=768, out_features=50304, bias=False)
)
```

```
prompt = "The capital of India is"

inputs = tokenizer(prompt, return_tensors="pt").to(model.device)

with torch.no_grad():
    output_ids = model.generate(
        **inputs,
        max_new_tokens=30,
        do_sample=True,
        temperature=0.7,
        top_p=0.9
    )
```

```
print(tokenizer.decode(output_ids[0], skip_special_tokens=True))
```

```
[transformers] Setting `pad_token_id` to `eos_token_id`:0 for open-end generation.
The capital of India is a small town with a large number of ethnic minorities. The town was named after a prominent Hindu ru
```

```
!pip install -q transformer-lens
```

```
import torch
from transformer_lens import HookedTransformer

device = "cuda" if torch.cuda.is_available() else "cpu"

model = HookedTransformer.from_pretrained(
    "EleutherAI/pythia-160m",
    device=device
)

model.eval()
```

Loading weights: 100% 148/148 [00:00<00:00, 305.22it/s]

```
Loaded pretrained model EleutherAI/pythia-160m into HookedTransformer
HookedTransformer(
  (embed): Embed()
  (hook_embed): HookPoint(name='hook_embed')
  (blocks): ModuleList(
    (0): TransformerBlock(
      (ln1): LayerNormPre(
        (hook_scale): HookPoint(name='blocks.0.ln1.hook_scale')
        (hook_normalized): HookPoint(name='blocks.0.ln1.hook_normalized')
      )
      (ln2): LayerNormPre(
        (hook_scale): HookPoint(name='blocks.0.ln2.hook_scale')
        (hook_normalized): HookPoint(name='blocks.0.ln2.hook_normalized')
      )
      (attn): Attention(
        (hook_k): HookPoint(name='blocks.0.attn.hook_k')
        (hook_q): HookPoint(name='blocks.0.attn.hook_q')
        (hook_v): HookPoint(name='blocks.0.attn.hook_v')
        (hook_z): HookPoint(name='blocks.0.attn.hook_z')
        (hook_attn_scores): HookPoint(name='blocks.0.attn.hook_attn_scores')
        (hook_pattern): HookPoint(name='blocks.0.attn.hook_pattern')
        (hook_result): HookPoint(name='blocks.0.attn.hook_result')
        (hook_rot_k): HookPoint(name='blocks.0.attn.hook_rot_k')
        (hook_rot_q): HookPoint(name='blocks.0.attn.hook_rot_q')
      )
      (mlp): MLP(
        (hook_pre): HookPoint(name='blocks.0.mlp.hook_pre')
        (hook_post): HookPoint(name='blocks.0.mlp.hook_post')
      )
      (hook_attn_in): HookPoint(name='blocks.0.hook_attn_in')
      (hook_q_input): HookPoint(name='blocks.0.hook_q_input')
      (hook_k_input): HookPoint(name='blocks.0.hook_k_input')
      (hook_v_input): HookPoint(name='blocks.0.hook_v_input')
      (hook_mlp_in): HookPoint(name='blocks.0.hook_mlp_in')
      (hook_attn_out): HookPoint(name='blocks.0.hook_attn_out')
      (hook_mlp_out): HookPoint(name='blocks.0.hook_mlp_out')
      (hook_resid_pre): HookPoint(name='blocks.0.hook_resid_pre')
      (hook_resid_post): HookPoint(name='blocks.0.hook_resid_post')
    )
    (1): TransformerBlock(
      (ln1): LayerNormPre(
        (hook_scale): HookPoint(name='blocks.1.ln1.hook_scale')
        (hook_normalized): HookPoint(name='blocks.1.ln1.hook_normalized')
      )
      (ln2): LayerNormPre(
        (hook_scale): HookPoint(name='blocks.1.ln2.hook_scale')
        (hook_normalized): HookPoint(name='blocks.1.ln2.hook_normalized')
      )
      (attn): Attention(
        (hook_k): HookPoint(name='blocks.1.attn.hook_k')
        (hook_q): HookPoint(name='blocks.1.attn.hook_q')
        (hook_v): HookPoint(name='blocks.1.attn.hook_v')
        (hook_z): HookPoint(name='blocks.1.attn.hook_z')
        (hook_attn_scores): HookPoint(name='blocks.1.attn.hook_attn_scores')
        (hook_pattern): HookPoint(name='blocks.1.attn.hook_pattern')
        (hook_result): HookPoint(name='blocks.1.attn.hook_result')
        (hook_rot_k): HookPoint(name='blocks.1.attn.hook_rot_k')
        (hook_rot_q): HookPoint(name='blocks.1.attn.hook_rot_q')
      )
      (mlp): MLP(
        (hook_pre): HookPoint(name='blocks.1.mlp.hook_pre')
        (hook_post): HookPoint(name='blocks.1.mlp.hook_post')
      )
      (hook_attn_in): HookPoint(name='blocks.1.hook_attn_in')
      (hook_q_input): HookPoint(name='blocks.1.hook_q_input')
      (hook_k_input): HookPoint(name='blocks.1.hook_k_input')
      (hook_v_input): HookPoint(name='blocks.1.hook_v_input')
      (hook_mlp_in): HookPoint(name='blocks.1.hook_mlp_in')
      (hook_attn_out): HookPoint(name='blocks.1.hook_attn_out')
      (hook_mlp_out): HookPoint(name='blocks.1.hook_mlp_out')
      (hook_resid_pre): HookPoint(name='blocks.1.hook_resid_pre')
      (hook_resid_post): HookPoint(name='blocks.1.hook_resid_post')
    )
    (2): TransformerBlock(
      (ln1): LayerNormPre(
        (hook_scale): HookPoint(name='blocks.2.ln1.hook_scale')
        (hook_normalized): HookPoint(name='blocks.2.ln1.hook_normalized')
      )
      (ln2): LayerNormPre(
        (hook_scale): HookPoint(name='blocks.2.ln2.hook_scale')
        (hook_normalized): HookPoint(name='blocks.2.ln2.hook_normalized')
      )
      (attn): Attention(
        (hook_k): HookPoint(name='blocks.2.attn.hook_k')
        (hook_q): HookPoint(name='blocks.2.attn.hook_q')
        (hook_v): HookPoint(name='blocks.2.attn.hook_v')
        (hook_z): HookPoint(name='blocks.2.attn.hook_z')
        (hook_attn_scores): HookPoint(name='blocks.2.attn.hook_attn_scores')
```

```
      (hook_pattern): HookPoint(name='blocks.2.attn.hook_pattern')
      (hook_result): HookPoint(name='blocks.2.attn.hook_result')
      (hook_rot_k): HookPoint(name='blocks.2.attn.hook_rot_k')
      (hook_rot_q): HookPoint(name='blocks.2.attn.hook_rot_q')
    )
    (mlp): MLP(
      (hook_pre): HookPoint(name='blocks.2.mlp.hook_pre')
      (hook_post): HookPoint(name='blocks.2.mlp.hook_post')
    )
    (hook_attn_in): HookPoint(name='blocks.2.hook_attn_in')
    (hook_q_input): HookPoint(name='blocks.2.hook_q_input')
    (hook_k_input): HookPoint(name='blocks.2.hook_k_input')
    (hook_v_input): HookPoint(name='blocks.2.hook_v_input')
    (hook_mlp_in): HookPoint(name='blocks.2.hook_mlp_in')
    (hook_attn_out): HookPoint(name='blocks.2.hook_attn_out')
    (hook_mlp_out): HookPoint(name='blocks.2.hook_mlp_out')
    (hook_resid_pre): HookPoint(name='blocks.2.hook_resid_pre')
    (hook_resid_post): HookPoint(name='blocks.2.hook_resid_post')
  )
  (3): TransformerBlock(
    (ln1): LayerNormPre(
      (hook_scale): HookPoint(name='blocks.3.ln1.hook_scale')
      (hook_normalized): HookPoint(name='blocks.3.ln1.hook_normalized')
    )
    (ln2): LayerNormPre(
      (hook_scale): HookPoint(name='blocks.3.ln2.hook_scale')
      (hook_normalized): HookPoint(name='blocks.3.ln2.hook_normalized')
    )
    (attn): Attention(
      (hook_k): HookPoint(name='blocks.3.attn.hook_k')
      (hook_q): HookPoint(name='blocks.3.attn.hook_q')
      (hook_v): HookPoint(name='blocks.3.attn.hook_v')
      (hook_z): HookPoint(name='blocks.3.attn.hook_z')
      (hook_attn_scores): HookPoint(name='blocks.3.attn.hook_attn_scores')
      (hook_pattern): HookPoint(name='blocks.3.attn.hook_pattern')
      (hook_result): HookPoint(name='blocks.3.attn.hook_result')
      (hook_rot_k): HookPoint(name='blocks.3.attn.hook_rot_k')
      (hook_rot_q): HookPoint(name='blocks.3.attn.hook_rot_q')
    )
    (mlp): MLP(
      (hook_pre): HookPoint(name='blocks.3.mlp.hook_pre')
      (hook_post): HookPoint(name='blocks.3.mlp.hook_post')
    )
    (hook_attn_in): HookPoint(name='blocks.3.hook_attn_in')
    (hook_q_input): HookPoint(name='blocks.3.hook_q_input')
    (hook_k_input): HookPoint(name='blocks.3.hook_k_input')
    (hook_v_input): HookPoint(name='blocks.3.hook_v_input')
    (hook_mlp_in): HookPoint(name='blocks.3.hook_mlp_in')
    (hook_attn_out): HookPoint(name='blocks.3.hook_attn_out')
    (hook_mlp_out): HookPoint(name='blocks.3.hook_mlp_out')
    (hook_resid_pre): HookPoint(name='blocks.3.hook_resid_pre')
    (hook_resid_post): HookPoint(name='blocks.3.hook_resid_post')
  )
  (4): TransformerBlock(
    (ln1): LayerNormPre(
      (hook_scale): HookPoint(name='blocks.4.ln1.hook_scale')
      (hook_normalized): HookPoint(name='blocks.4.ln1.hook_normalized')
    )
    (ln2): LayerNormPre(
      (hook_scale): HookPoint(name='blocks.4.ln2.hook_scale')
      (hook_normalized): HookPoint(name='blocks.4.ln2.hook_normalized')
    )
    (attn): Attention(
      (hook_k): HookPoint(name='blocks.4.attn.hook_k')
      (hook_q): HookPoint(name='blocks.4.attn.hook_q')
      (hook_v): HookPoint(name='blocks.4.attn.hook_v')
      (hook_z): HookPoint(name='blocks.4.attn.hook_z')
      (hook_attn_scores): HookPoint(name='blocks.4.attn.hook_attn_scores')
      (hook_pattern): HookPoint(name='blocks.4.attn.hook_pattern')
      (hook_result): HookPoint(name='blocks.4.attn.hook_result')
      (hook_rot_k): HookPoint(name='blocks.4.attn.hook_rot_k')
      (hook_rot_q): HookPoint(name='blocks.4.attn.hook_rot_q')
    )
    (mlp): MLP(
      (hook_pre): HookPoint(name='blocks.4.mlp.hook_pre')
      (hook_post): HookPoint(name='blocks.4.mlp.hook_post')
    )
    (hook_attn_in): HookPoint(name='blocks.4.hook_attn_in')
    (hook_q_input): HookPoint(name='blocks.4.hook_q_input')
    (hook_k_input): HookPoint(name='blocks.4.hook_k_input')
    (hook_v_input): HookPoint(name='blocks.4.hook_v_input')
    (hook_mlp_in): HookPoint(name='blocks.4.hook_mlp_in')
    (hook_attn_out): HookPoint(name='blocks.4.hook_attn_out')
    (hook_mlp_out): HookPoint(name='blocks.4.hook_mlp_out')
    (hook_resid_pre): HookPoint(name='blocks.4.hook_resid_pre')
    (hook_resid_post): HookPoint(name='blocks.4.hook_resid_post')
  )
  (5): TransformerBlock(
    (ln1): LayerNormPre(
      (hook_scale): HookPoint(name='blocks.5.ln1.hook_scale')
      (hook_normalized): HookPoint(name='blocks.5.ln1.hook_normalized')
```

```
      )
      (ln2): LayerNormPre(
        (hook_scale): HookPoint(name='blocks.5.ln2.hook_scale')
        (hook_normalized): HookPoint(name='blocks.5.ln2.hook_normalized')
      )
      (attn): Attention(
        (hook_k): HookPoint(name='blocks.5.attn.hook_k')
        (hook_q): HookPoint(name='blocks.5.attn.hook_q')
        (hook_v): HookPoint(name='blocks.5.attn.hook_v')
        (hook_z): HookPoint(name='blocks.5.attn.hook_z')
        (hook_attn_scores): HookPoint(name='blocks.5.attn.hook_attn_scores')
        (hook_pattern): HookPoint(name='blocks.5.attn.hook_pattern')
        (hook_result): HookPoint(name='blocks.5.attn.hook_result')
        (hook_rot_k): HookPoint(name='blocks.5.attn.hook_rot_k')
        (hook_rot_q): HookPoint(name='blocks.5.attn.hook_rot_q')
      )
      (mlp): MLP(
        (hook_pre): HookPoint(name='blocks.5.mlp.hook_pre')
        (hook_post): HookPoint(name='blocks.5.mlp.hook_post')
      )
      (hook_attn_in): HookPoint(name='blocks.5.hook_attn_in')
      (hook_q_input): HookPoint(name='blocks.5.hook_q_input')
      (hook_k_input): HookPoint(name='blocks.5.hook_k_input')
      (hook_v_input): HookPoint(name='blocks.5.hook_v_input')
      (hook_mlp_in): HookPoint(name='blocks.5.hook_mlp_in')
      (hook_attn_out): HookPoint(name='blocks.5.hook_attn_out')
      (hook_mlp_out): HookPoint(name='blocks.5.hook_mlp_out')
      (hook_resid_pre): HookPoint(name='blocks.5.hook_resid_pre')
      (hook_resid_post): HookPoint(name='blocks.5.hook_resid_post')
    )
    (6): TransformerBlock(
      (ln1): LayerNormPre(
        (hook_scale): HookPoint(name='blocks.6.ln1.hook_scale')
        (hook_normalized): HookPoint(name='blocks.6.ln1.hook_normalized')
      )
      (ln2): LayerNormPre(
        (hook_scale): HookPoint(name='blocks.6.ln2.hook_scale')
        (hook_normalized): HookPoint(name='blocks.6.ln2.hook_normalized')
      )
      (attn): Attention(
        (hook_k): HookPoint(name='blocks.6.attn.hook_k')
        (hook_q): HookPoint(name='blocks.6.attn.hook_q')
        (hook_v): HookPoint(name='blocks.6.attn.hook_v')
        (hook_z): HookPoint(name='blocks.6.attn.hook_z')
        (hook_attn_scores): HookPoint(name='blocks.6.attn.hook_attn_scores')
        (hook_pattern): HookPoint(name='blocks.6.attn.hook_pattern')
        (hook_result): HookPoint(name='blocks.6.attn.hook_result')
        (hook_rot_k): HookPoint(name='blocks.6.attn.hook_rot_k')
        (hook_rot_q): HookPoint(name='blocks.6.attn.hook_rot_q')
      )
      (mlp): MLP(
        (hook_pre): HookPoint(name='blocks.6.mlp.hook_pre')
        (hook_post): HookPoint(name='blocks.6.mlp.hook_post')
      )
      (hook_attn_in): HookPoint(name='blocks.6.hook_attn_in')
      (hook_q_input): HookPoint(name='blocks.6.hook_q_input')
      (hook_k_input): HookPoint(name='blocks.6.hook_k_input')
      (hook_v_input): HookPoint(name='blocks.6.hook_v_input')
      (hook_mlp_in): HookPoint(name='blocks.6.hook_mlp_in')
      (hook_attn_out): HookPoint(name='blocks.6.hook_attn_out')
      (hook_mlp_out): HookPoint(name='blocks.6.hook_mlp_out')
      (hook_resid_pre): HookPoint(name='blocks.6.hook_resid_pre')
      (hook_resid_post): HookPoint(name='blocks.6.hook_resid_post')
    )
    (7): TransformerBlock(
      (ln1): LayerNormPre(
        (hook_scale): HookPoint(name='blocks.7.ln1.hook_scale')
        (hook_normalized): HookPoint(name='blocks.7.ln1.hook_normalized')
      )
      (ln2): LayerNormPre(
        (hook_scale): HookPoint(name='blocks.7.ln2.hook_scale')
        (hook_normalized): HookPoint(name='blocks.7.ln2.hook_normalized')
      )
      (attn): Attention(
        (hook_k): HookPoint(name='blocks.7.attn.hook_k')
        (hook_q): HookPoint(name='blocks.7.attn.hook_q')
        (hook_v): HookPoint(name='blocks.7.attn.hook_v')
        (hook_z): HookPoint(name='blocks.7.attn.hook_z')
        (hook_attn_scores): HookPoint(name='blocks.7.attn.hook_attn_scores')
        (hook_pattern): HookPoint(name='blocks.7.attn.hook_pattern')
        (hook_result): HookPoint(name='blocks.7.attn.hook_result')
        (hook_rot_k): HookPoint(name='blocks.7.attn.hook_rot_k')
        (hook_rot_q): HookPoint(name='blocks.7.attn.hook_rot_q')
      )
      (mlp): MLP(
        (hook_pre): HookPoint(name='blocks.7.mlp.hook_pre')
        (hook_post): HookPoint(name='blocks.7.mlp.hook_post')
      )
      (hook_attn_in): HookPoint(name='blocks.7.hook_attn_in')
      (hook_q_input): HookPoint(name='blocks.7.hook_q_input')
```

```
      (hook_k_input): HookPoint(name='blocks.7.hook_k_input')
      (hook_v_input): HookPoint(name='blocks.7.hook_v_input')
      (hook_mlp_in): HookPoint(name='blocks.7.hook_mlp_in')
      (hook_attn_out): HookPoint(name='blocks.7.hook_attn_out')
      (hook_mlp_out): HookPoint(name='blocks.7.hook_mlp_out')
      (hook_resid_pre): HookPoint(name='blocks.7.hook_resid_pre')
      (hook_resid_post): HookPoint(name='blocks.7.hook_resid_post')
    )
    (8): TransformerBlock(
      (ln1): LayerNormPre(
        (hook_scale): HookPoint(name='blocks.8.ln1.hook_scale')
        (hook_normalized): HookPoint(name='blocks.8.ln1.hook_normalized')
      )
      (ln2): LayerNormPre(
        (hook_scale): HookPoint(name='blocks.8.ln2.hook_scale')
        (hook_normalized): HookPoint(name='blocks.8.ln2.hook_normalized')
      )
      (attn): Attention(
        (hook_k): HookPoint(name='blocks.8.attn.hook_k')
        (hook_q): HookPoint(name='blocks.8.attn.hook_q')
        (hook_v): HookPoint(name='blocks.8.attn.hook_v')
        (hook_z): HookPoint(name='blocks.8.attn.hook_z')
        (hook_attn_scores): HookPoint(name='blocks.8.attn.hook_attn_scores')
        (hook_pattern): HookPoint(name='blocks.8.attn.hook_pattern')
        (hook_result): HookPoint(name='blocks.8.attn.hook_result')
        (hook_rot_k): HookPoint(name='blocks.8.attn.hook_rot_k')
        (hook_rot_q): HookPoint(name='blocks.8.attn.hook_rot_q')
      )
      (mlp): MLP(
        (hook_pre): HookPoint(name='blocks.8.mlp.hook_pre')
        (hook_post): HookPoint(name='blocks.8.mlp.hook_post')
      )
      (hook_attn_in): HookPoint(name='blocks.8.hook_attn_in')
      (hook_q_input): HookPoint(name='blocks.8.hook_q_input')
      (hook_k_input): HookPoint(name='blocks.8.hook_k_input')
      (hook_v_input): HookPoint(name='blocks.8.hook_v_input')
      (hook_mlp_in): HookPoint(name='blocks.8.hook_mlp_in')
      (hook_attn_out): HookPoint(name='blocks.8.hook_attn_out')
      (hook_mlp_out): HookPoint(name='blocks.8.hook_mlp_out')
      (hook_resid_pre): HookPoint(name='blocks.8.hook_resid_pre')
      (hook_resid_post): HookPoint(name='blocks.8.hook_resid_post')
    )
    (9): TransformerBlock(
      (ln1): LayerNormPre(
        (hook_scale): HookPoint(name='blocks.9.ln1.hook_scale')
        (hook_normalized): HookPoint(name='blocks.9.ln1.hook_normalized')
      )
      (ln2): LayerNormPre(
        (hook_scale): HookPoint(name='blocks.9.ln2.hook_scale')
        (hook_normalized): HookPoint(name='blocks.9.ln2.hook_normalized')
      )
      (attn): Attention(
        (hook_k): HookPoint(name='blocks.9.attn.hook_k')
        (hook_q): HookPoint(name='blocks.9.attn.hook_q')
        (hook_v): HookPoint(name='blocks.9.attn.hook_v')
        (hook_z): HookPoint(name='blocks.9.attn.hook_z')
        (hook_attn_scores): HookPoint(name='blocks.9.attn.hook_attn_scores')
        (hook_pattern): HookPoint(name='blocks.9.attn.hook_pattern')
        (hook_result): HookPoint(name='blocks.9.attn.hook_result')
        (hook_rot_k): HookPoint(name='blocks.9.attn.hook_rot_k')
        (hook_rot_q): HookPoint(name='blocks.9.attn.hook_rot_q')
      )
      (mlp): MLP(
        (hook_pre): HookPoint(name='blocks.9.mlp.hook_pre')
        (hook_post): HookPoint(name='blocks.9.mlp.hook_post')
      )
      (hook_attn_in): HookPoint(name='blocks.9.hook_attn_in')
      (hook_q_input): HookPoint(name='blocks.9.hook_q_input')
      (hook_k_input): HookPoint(name='blocks.9.hook_k_input')
      (hook_v_input): HookPoint(name='blocks.9.hook_v_input')
      (hook_mlp_in): HookPoint(name='blocks.9.hook_mlp_in')
      (hook_attn_out): HookPoint(name='blocks.9.hook_attn_out')
      (hook_mlp_out): HookPoint(name='blocks.9.hook_mlp_out')
      (hook_resid_pre): HookPoint(name='blocks.9.hook_resid_pre')
      (hook_resid_post): HookPoint(name='blocks.9.hook_resid_post')
    )
    (10): TransformerBlock(
      (ln1): LayerNormPre(
        (hook_scale): HookPoint(name='blocks.10.ln1.hook_scale')
        (hook_normalized): HookPoint(name='blocks.10.ln1.hook_normalized')
      )
      (ln2): LayerNormPre(
        (hook_scale): HookPoint(name='blocks.10.ln2.hook_scale')
        (hook_normalized): HookPoint(name='blocks.10.ln2.hook_normalized')
      )
      (attn): Attention(
        (hook_k): HookPoint(name='blocks.10.attn.hook_k')
        (hook_q): HookPoint(name='blocks.10.attn.hook_q')
        (hook_v): HookPoint(name='blocks.10.attn.hook_v')
        (hook_z): HookPoint(name='blocks.10.attn.hook_z')
        (hook_attn_scores): HookPoint(name='blocks.10.attn.hook_attn_scores')
```

```
text = "def add_two_numbers(a, b):"
tokens = model.to_tokens(text)

with torch.no_grad():
    logits = model(tokens)

print(logits.shape)
```

```
torch.Size([1, 11, 50304])
```

Double-click (or enter) to edit

```
text = "def add_two_numbers(a, b):\n    return a + b"

tokens = model.to_tokens(text)

with torch.no_grad():
    logits, cache = model.run_with_cache(tokens)

activation = cache["blocks.6.hook_resid_post"]

print(activation.shape)
```

```
torch.Size([1, 17, 768])
```

```
acts = activation.reshape(-1, activation.shape[-1])
print("SAE training input shape:", acts.shape)
```

```
SAE training input shape: torch.Size([17, 768])
```

```
!pip install -q transformer-lens datasets einops tqdm
```

```
import os
import torch
import torch.nn as nn
import torch.nn.functional as F

from tqdm import tqdm
from datasets import load_dataset
from torch.utils.data import TensorDataset, DataLoader
from transformer_lens import HookedTransformer

device = "cuda" if torch.cuda.is_available() else "cpu"
print("Using device:", device)

MODEL_NAME = "EleutherAI/pythia-160m"
HOOK_NAME = "blocks.6.hook_resid_post"
LAYER = 6

MAX_LENGTH = 128
MAX_ACTIVATIONS = 50_000
BATCH_SIZE_ACT_COLLECTION = 1

D_IN = 768
D_SAE = 4096
SAE_BATCH_SIZE = 512
LR = 1e-3
L1_COEFF = 3e-4
EPOCHS = 10

SAVE_PATH = "baseline_pythia160m_layer6_sae.pt"
```

```
Using device: cuda
```

```
model = HookedTransformer.from_pretrained(
    MODEL_NAME,
    device=device
)

model.eval()

for param in model.parameters():
    param.requires_grad = False

print("Loaded frozen Pythia-160M")
```

```
Loading weights: 100%                                   148/148 [00:00<00:00, 450.04it/s]
Loaded pretrained model EleutherAI/pythia-160m into HookedTransformer
Loaded frozen Pythia-160M
```

```
dataset = load_dataset("NeelNanda/pile-10k", split="train")

print(dataset)
print(dataset[0]["text"][:500])
```

```
README.md: 100%                                   373/373 [00:00<00:00, 22.2kB/s]

dataset_infos.json: 100%                          921/921 [00:00<00:00, 95.5kB/s]

data/train-00000-of-00001-4746b8785c874c(…): 100%                33.3M/33.3M [00:00<00:00, 46.7MB/s]

Generating train split: 100%                      10000/10000 [00:00<00:00, 22255.03 examples/s]
Dataset({
    features: ['text', 'meta'],
    num_rows: 10000
})
It is done, and submitted. You can play “Survival of the Tastiest” on Android, and on the web. Playing on the web works, but

There’s a lot I’d like to talk about. I’ll go through every topic, insted of making the typical what went right/wrong list.

Concept

Working over the theme was probably one of the hardest tasks I had to face.

Originally, I had an idea of what kind of game I wanted to develop, gameplay wis
```

```
all_acts = []
all_token_strs = []

with torch.no_grad():
    for item in tqdm(dataset):
        text = item["text"]

        if not isinstance(text, str) or len(text.strip()) == 0:
            continue

        tokens = model.to_tokens(
            text,
            truncate=True,
            prepend_bos=True
        ).to(device)

        tokens = tokens[:, :MAX_LENGTH]

        logits, cache = model.run_with_cache(tokens)

        acts = cache[HOOK_NAME]

        acts = acts.reshape(-1, acts.shape[-1])

        token_ids = tokens.reshape(-1).detach().cpu().tolist()
        token_strs = model.to_str_tokens(tokens[0])


        acts = acts[1:]
        token_strs = token_strs[1:]

        all_acts.append(acts.detach().cpu().float())
        all_token_strs.extend(token_strs)

        total = sum(x.shape[0] for x in all_acts)

        if total >= MAX_ACTIVATIONS:
            break

activations = torch.cat(all_acts, dim=0)[:MAX_ACTIVATIONS]
all_token_strs = all_token_strs[:MAX_ACTIVATIONS]

print("Activation tensor:", activations.shape)
print("Number of token strings:", len(all_token_strs))
print("First 20 tokens:", all_token_strs[:20])
```

```
  4%|▌          | 411/10000 [00:45<17:50,  8.95it/s]Activation tensor: torch.Size([50000, 768])
Number of token strings: 50000
First 20 tokens: ['It', ' is', ' done', ',', ' and', ' submitted', '.', ' You', ' can', ' play', ' “', 'Sur', 'vival', ' of'
```

```
class SparseAutoencoder(nn.Module):
    def __init__(self, d_in, d_sae):
        super().__init__()

        self.d_in = d_in
        self.d_sae = d_sae

        self.W_enc = nn.Parameter(torch.empty(d_in, d_sae))
        self.b_enc = nn.Parameter(torch.zeros(d_sae))

        self.W_dec = nn.Parameter(torch.empty(d_sae, d_in))
        self.b_dec = nn.Parameter(torch.zeros(d_in))

        self.reset_parameters()

    def reset_parameters(self):
        nn.init.kaiming_uniform_(self.W_enc)
        nn.init.kaiming_uniform_(self.W_dec)

        with torch.no_grad():
            self.W_dec.data = self.W_dec.data / self.W_dec.data.norm(dim=1, keepdim=True).clamp(min=1e-6)

    def encode(self, x):

        pre_acts = x @ self.W_enc + self.b_enc
        features = F.relu(pre_acts)
        return features

    def decode(self, features):

        return features @ self.W_dec + self.b_dec

    def forward(self, x):
        features = self.encode(x)
        reconstruction = self.decode(features)
        return reconstruction, features

    @torch.no_grad()
    def normalize_decoder(self):

        self.W_dec.data = self.W_dec.data / self.W_dec.data.norm(dim=1, keepdim=True).clamp(min=1e-6)
```

```
sae = SparseAutoencoder(d_in=D_IN, d_sae=D_SAE).to(device)

train_loader = DataLoader(
    TensorDataset(activations),
    batch_size=SAE_BATCH_SIZE,
    shuffle=True,
    drop_last=True
)

optimizer = torch.optim.Adam(sae.parameters(), lr=LR)

for epoch in range(EPOCHS):
    sae.train()

    total_loss = 0.0
    total_mse = 0.0
    total_l1 = 0.0
    total_l0 = 0.0

    for batch in tqdm(train_loader, desc=f"Epoch {epoch+1}/{EPOCHS}"):
        x = batch[0].to(device)

        reconstruction, features = sae(x)

        mse_loss = F.mse_loss(reconstruction, x)
        l1_loss = features.abs().sum(dim=-1).mean()

        loss = mse_loss + L1_COEFF * l1_loss

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        sae.normalize_decoder()

        with torch.no_grad():
            l0 = (features > 0).float().sum(dim=-1).mean()
```

```
        total_loss += loss.item()
        total_mse += mse_loss.item()
        total_l1 += l1_loss.item()
        total_l0 += l0.item()

    n = len(train_loader)

    print(
        f"Epoch {epoch+1} | "
        f"Loss: {total_loss/n:.6f} | "
        f"MSE: {total_mse/n:.6f} | "
        f"L1: {total_l1/n:.6f} | "
        f"Avg L0: {total_l0/n:.2f}"
    )
```

```
Epoch 1/10: 100%|██████████| 97/97 [00:01<00:00, 92.42it/s]
Epoch 1 | Loss: 0.342316 | MSE: 0.279342 | L1: 209.911722 | Avg L0: 514.27
Epoch 2/10: 100%|██████████| 97/97 [00:00<00:00, 124.05it/s]
Epoch 2 | Loss: 0.140271 | MSE: 0.094407 | L1: 152.880604 | Avg L0: 360.22
Epoch 3/10: 100%|██████████| 97/97 [00:00<00:00, 123.57it/s]
Epoch 3 | Loss: 0.115239 | MSE: 0.068441 | L1: 155.991332 | Avg L0: 391.07
Epoch 4/10: 100%|██████████| 97/97 [00:00<00:00, 124.78it/s]
Epoch 4 | Loss: 0.102850 | MSE: 0.056279 | L1: 155.237852 | Avg L0: 414.60
Epoch 5/10: 100%|██████████| 97/97 [00:00<00:00, 122.22it/s]
Epoch 5 | Loss: 0.094731 | MSE: 0.048739 | L1: 153.304919 | Avg L0: 432.08
Epoch 6/10: 100%|██████████| 97/97 [00:00<00:00, 123.82it/s]
Epoch 6 | Loss: 0.089246 | MSE: 0.043864 | L1: 151.273245 | Avg L0: 445.07
Epoch 7/10: 100%|██████████| 97/97 [00:00<00:00, 123.91it/s]
Epoch 7 | Loss: 0.085037 | MSE: 0.040039 | L1: 149.993165 | Avg L0: 456.27
Epoch 8/10: 100%|██████████| 97/97 [00:00<00:00, 122.38it/s]
Epoch 8 | Loss: 0.082106 | MSE: 0.037447 | L1: 148.862556 | Avg L0: 464.70
Epoch 9/10: 100%|██████████| 97/97 [00:00<00:00, 110.79it/s]
Epoch 9 | Loss: 0.079831 | MSE: 0.035514 | L1: 147.723957 | Avg L0: 470.84
Epoch 10/10: 100%|██████████| 97/97 [00:01<00:00, 82.98it/s] Epoch 10 | Loss: 0.078275 | MSE: 0.034391 | L1: 146.279865 | Av
```

```
save_dict = {
    "sae_state_dict": sae.state_dict(),
    "model_name": MODEL_NAME,
    "hook_name": HOOK_NAME,
    "layer": LAYER,
    "d_in": D_IN,
    "d_sae": D_SAE,
    "max_length": MAX_LENGTH,
    "max_activations": MAX_ACTIVATIONS,
    "l1_coeff": L1_COEFF,
    "lr": LR,
    "epochs": EPOCHS,
}

torch.save(save_dict, SAVE_PATH)

print(f"Saved SAE to {SAVE_PATH}")
```

```
Saved SAE to baseline_pythia160m_layer6_sae.pt
```

```
@torch.no_grad()
def get_top_activating_tokens(sae, activations, token_strs, feature_idx, top_k=20, batch_size=1024):
    sae.eval()

    all_scores = []

    for start in range(0, activations.shape[0], batch_size):
        end = start + batch_size
        x = activations[start:end].to(device)

        features = sae.encode(x)
        scores = features[:, feature_idx].detach().cpu()

        all_scores.append(scores)

    all_scores = torch.cat(all_scores, dim=0)

    top_scores, top_indices = torch.topk(all_scores, k=top_k)

    results = []

    for score, idx in zip(top_scores, top_indices):
        idx = idx.item()
        results.append({
            "activation": score.item(),
            "token": token_strs[idx],
```

```
            "index": idx,
        })

    return results
```

```
feature_idx = 0

top_tokens = get_top_activating_tokens(
    sae=sae,
    activations=activations,
    token_strs=all_token_strs,
    feature_idx=feature_idx,
    top_k=20
)

for row in top_tokens:
    print(f"{row['activation']:.4f}  |  {repr(row['token'])}")
```

```
0.3705  |  '\n'
0.3566  |  '\n'
0.3437  |  '\n'
0.3374  |  '\n'
0.3051  |  '\n'
0.2805  |  '\n'
0.2439  |  '\n'
0.2128  |  '\n'
0.1244  |  '\n'
0.1090  |  '\n'
0.1002  |  '\n'
0.0549  |  '\n'
0.0320  |  '\n'
0.0078  |  ' consent'
0.0000  |  ' T'
0.0000  |  ' You'
0.0000  |  'ast'
0.0000  |  'iest'
0.0000  |  ','
0.0000  |  ' can'
```

```
for feature_idx in [0, 1, 2, 10, 50, 100, 500]:
    print("=" * 80)
    print("Feature:", feature_idx)

    top_tokens = get_top_activating_tokens(
        sae=sae,
        activations=activations,
        token_strs=all_token_strs,
        feature_idx=feature_idx,
        top_k=10
    )

    for row in top_tokens:
        print(f"{row['activation']:.4f}  |  {repr(row['token'])}")
```

```
0.9589  |  ','
0.9231  |  ' Intel'
0.9032  |  ' &'
0.8624  |  ' rubbed'
0.8413  |  ' Manit'
0.8299  |  '-'
```

```
0.1932  |  '-'
0.1929  |  ' select'
================================================================================
Feature: 100
0.4248  |  '-'
0.4233  |  ' took'
0.2790  |  ' take'
0.2637  |  'CO'
0.2062  |  '\n'
0.1962  |  ' Henderson'
0.1863  |  'return'
0.1600  |  '-'
0.1362  |  'th'
0.1156  |  ' Conf'
================================================================================
Feature: 500
1.3053  |  ' nanot'
1.2573  |  ' quint'
1.2177  |  ' on'
1.1894  |  ' bee'
1.1161  |  'oc'
1.0975  |  ' bee'
1.0848  |  ' pro'
1.0800  |  ' plane'
1.0750  |  'order'
1.0728  |  'oc'
```

```python
@torch.no_grad()
def get_feature_stats(sae, activations, batch_size=1024):
    sae.eval()

    feature_sums = torch.zeros(sae.d_sae)
    feature_counts = torch.zeros(sae.d_sae)
    feature_max = torch.zeros(sae.d_sae)

    for start in tqdm(range(0, activations.shape[0], batch_size)):
        end = start + batch_size
        x = activations[start:end].to(device)

        features = sae.encode(x).detach().cpu()

        feature_sums += features.sum(dim=0)
        feature_counts += (features > 0).float().sum(dim=0)
        feature_max = torch.maximum(feature_max, features.max(dim=0).values)

    firing_rate = feature_counts / activations.shape[0]
    mean_activation = feature_sums / activations.shape[0]

    return {
        "firing_rate": firing_rate,
        "mean_activation": mean_activation,
        "max_activation": feature_max,
    }

stats = get_feature_stats(sae, activations)

print("Dead features:", (stats["firing_rate"] == 0).sum().item())
print("Features firing > 1%:", (stats["firing_rate"] > 0.01).sum().item())
print("Features firing > 10%:", (stats["firing_rate"] > 0.10).sum().item())
```

```
100%|██████████| 49/49 [00:01<00:00, 28.82it/s]Dead features: 159
Features firing > 1%: 1326
Features firing > 10%: 1074
```

```python
top_feature_scores, top_feature_indices = torch.topk(stats["max_activation"], k=20)

for score, idx in zip(top_feature_scores, top_feature_indices):
    print(f"Feature {idx.item()} | max activation {score.item():.4f}")
```

```
Feature 3921 | max activation 77.7725
Feature 1527 | max activation 74.3296
Feature 3784 | max activation 71.0540
Feature 384 | max activation 62.4416
Feature 648 | max activation 62.1587
Feature 2860 | max activation 60.5561
Feature 2705 | max activation 57.8966
Feature 3472 | max activation 35.5869
Feature 3138 | max activation 31.3331
Feature 3028 | max activation 28.5235
Feature 2811 | max activation 28.1890
Feature 3097 | max activation 27.5869
Feature 687 | max activation 24.3910
Feature 2414 | max activation 21.8648
Feature 3889 | max activation 20.8912
Feature 3167 | max activation 20.7808
```

```
Feature 137 | max activation 18.0530
Feature 1933 | max activation 17.9309
Feature 2939 | max activation 15.6932
Feature 2214 | max activation 15.0130
```

```
feature_idx = top_feature_indices[0].item()

top_tokens = get_top_activating_tokens(
    sae=sae,
    activations=activations,
    token_strs=all_token_strs,
    feature_idx=feature_idx,
    top_k=20
)

print("Feature:", feature_idx)
for row in top_tokens:
    print(f"{row['activation']:.4f}  |  {repr(row['token'])}")
```

```
Feature: 3921
77.7725  |  '\n'
77.3135  |  '\n'
77.2717  |  '.'
77.1042  |  '\n'
76.9869  |  '.'
76.9143  |  '.'
76.9143  |  '.'
76.9143  |  '.'
76.9143  |  '.'
76.9143  |  '.'
76.7855  |  '\n'
76.6974  |  '.'
76.6307  |  '.'
76.6307  |  '.'
76.6130  |  '\n'
76.5310  |  '\n'
76.4906  |  '\n'
76.4476  |  '.'
76.4206  |  '\n'
76.4166  |  '.'
```

```
torch.save(
    {
        "firing_rate": stats["firing_rate"],
        "mean_activation": stats["mean_activation"],
        "max_activation": stats["max_activation"],
        "token_strs": all_token_strs,
    },
    "baseline_sae_feature_stats.pt"
)

print("Saved feature stats")
```

```
Saved feature stats
```

```
def show_token_context(token_strs, center_idx, window=8):
    start = max(0, center_idx - window)
    end = min(len(token_strs), center_idx + window + 1)

    context = token_strs[start:end]

    center_local_idx = center_idx - start
    context[center_local_idx] = f"[[{context[center_local_idx]}]]"

    return "".join(context)
```

```
feature_idx = 500

top_tokens = get_top_activating_tokens(
    sae=sae,
    activations=activations,
    token_strs=all_token_strs,
    feature_idx=feature_idx,
    top_k=10
)

for row in top_tokens:
    print("=" * 80)
    print("Activation:", row["activation"])
    print("Token:", repr(row["token"]))
    print(show_token_context(all_token_strs, row["index"], window=12))
```

```
================================================================================
Activation: 1.3052825927734375
Token: ' nanot'
 implant considerations. Specifically, the effects of TiO(2)[[ nanot]]ube surfaces for bone regeneration will be discussed.
================================================================================
Activation: 1.257307529449463
Token: ' quint'
 by using the postal codes. The distribution of the SIMD[[ quint]]iles was examined to determine the possible influence of c
================================================================================
Activation: 1.2176507711410522
Token: ' on'
 then you’re in good company! If not, carry[[ on]]!

This mischievousness reminds me of my
================================================================================
Activation: 1.1893936395645142
Token: ' bee'
keeping latest news. On the other hand if you are starting[[ bee]]keeping and would like to begin professional beekeeping to
================================================================================
Activation: 1.1161470413208008
Token: 'oc'
ary power for space probes and satellitesPrevalence of varic[[oc]]oele and its association with body mass index among 39,
================================================================================
Activation: 1.0975475311279297
Token: ' bee'
 if you are starting beekeeping and would like to begin professional[[ bee]]keeping today download a copy of our beekeeping
================================================================================
Activation: 1.084833025932312
Token: ' pro'
 mi búsqueda por aprender programación por mis[[ pro]]pios medios, me he topado con el tem
================================================================================
Activation: 1.080033779144287
Token: ' plane'
 next haunts Chicago to this day.

As the[[ plane]] plunged downward, it kept rotating — past the point of perpendicular
================================================================================
Activation: 1.0750290155410767
Token: 'order'
Recorder` and `KinesisFirehoseRec[[order]]` allow you to interface with Amazon Kinesis Data St
================================================================================
Activation: 1.0727869272232056
Token: 'oc'
based cross-sectional study to evaluate the prevalence of varic[[oc]]oele among rural men in eastern China and its associati
```

```python
baseline_feature_notes = {
    500: {
        "label": "paired alternative / binary relation feature",
        "evidence_tokens": [" both", " neither", " either", " Both"],
        "confidence": "medium",
        "notes": "Mostly activates on both/either/neither contexts, but has some noisy examples such as String."
    },
    100: {
        "label": "uppercase S token feature",
        "evidence_tokens": ["S"],
        "confidence": "high",
        "notes": "Strongly activates on repeated uppercase S tokens."
    },
    3138: {
        "label": "newline / period formatting feature",
        "evidence_tokens": ["\\n", "."],
        "confidence": "medium",
        "notes": "Activates on sentence or paragraph boundary-like tokens."
    }
}
```

```python
import json

with open("baseline_feature_notes.json", "w") as f:
    json.dump(baseline_feature_notes, f, indent=2)

print("Saved baseline feature notes.")
```

```
Saved baseline feature notes.
```

```python
!pip install -q transformer-lens datasets tqdm einops
```

```python
import torch
import torch.nn.functional as F
from torch.utils.data import DataLoader
from datasets import load_dataset
from tqdm import tqdm
from transformer_lens import HookedTransformer
```

```
device = "cuda" if torch.cuda.is_available() else "cpu"
print("Using device:", device)

MODEL_NAME = "EleutherAI/pythia-160m"

MAX_LENGTH = 128
FT_BATCH_SIZE = 2
FT_LR = 1e-5
FT_STEPS = 500
GRAD_ACCUM_STEPS = 4

SAVE_FT_MODEL_PATH = "pythia160m_code_finetuned.pt"
```

```
Using device: cuda
```

```
ft_model = HookedTransformer.from_pretrained(
    MODEL_NAME,
    device=device
)

ft_model.train()

print("Loaded Pythia-160M for fine-tuning")
```

Loading weights: 100% 148/148 [00:00<00:00, 360.34it/s]

```
Loaded pretrained model EleutherAI/pythia-160m into HookedTransformer
Loaded Pythia-160M for fine-tuning
```

```
ft_model = HookedTransformer.from_pretrained(
    MODEL_NAME,
    device=device
)

ft_model.train()

print("Loaded Pythia-160M for fine-tuning")
```

Loading weights: 100% 148/148 [00:00<00:00, 529.66it/s]

```
Loaded pretrained model EleutherAI/pythia-160m into HookedTransformer
Loaded Pythia-160M for fine-tuning
```

```
from datasets import load_dataset

code_dataset = load_dataset(
    "claudios/code_search_net",
    "python",
    split="train",
    streaming=True
)

print(code_dataset)
```

README.md: 100% 13.6k/13.6k [00:00<00:00, 581kB/s]

```
IterableDataset({
    features: ['repository_name', 'func_path_in_repository', 'func_name', 'whole_func_string', 'language', 'func_code_string
    num_shards: 3
})
```

```
def code_text_iterator(dataset, max_items=5000):
    count = 0

    for item in dataset:
        code = item.get("func_code_string", None)

        if code is None:
            code = item.get("whole_func_string", None)

        if code is None:
            continue

        if not isinstance(code, str):
            continue

        if len(code.strip()) < 50:
            continue

        yield code
```

```
        count += 1
        if count >= max_items:
            break
```

```
sample_codes = list(code_text_iterator(code_dataset, max_items=5))

for i, code in enumerate(sample_codes):
    print("=" * 80)
    print("Sample", i)
    print(code[:500])
```

```
==================================================

ix, recursive = True):
to this element's ID, and optionally to all child IDs as well. There is sually no need to call this directly, invoked implici
+= idsuffix



uffix(idsuffix, recursive)
tion:

==================================================


t relations for elements within the scop. There is sually no need to call this directly, invoked implicitly by :meth:`copy`"'

 AbstractElement):
elf
()
==================================================


ocument. Usually no need to call this directly, invoked implicitly by :meth:`copy`"""

.id:
elf.id] = self

 AbstractElement):
doc)
==================================================

t',strict=True, correctionhandling=CorrectionHandling.CURRENT): #pylint: disable=too-many-return-statements
 have text (of the specified class)

ke :meth:`text`, this checks strictly, i.e. the element itself must have the text and it is not inherited from its children.


lass of the text content to obtain, defaults to ``current``.
Set this
==================================================

t',strict=True,correctionhandling=CorrectionHandling.CURRENT): #pylint: disable=too-many-return-statements
 have phonetic content (of the specified class)

ke :meth:`phon`, this checks strictly, i.e. the element itself must have the phonetic content and it is not inherited from it


lass of the phonetic content to obtain, defaults to ``current``.
```

```
code_dataset = load_dataset(
    "claudios/code_search_net",
    "python",
    split="train",
    streaming=True
)

code_texts = list(code_text_iterator(code_dataset, max_items=3000))

print("Number of code samples:", len(code_texts))
print(code_texts[0][:300])
```

```
Number of code samples: 3000
def addidsuffix(self, idsuffix, recursive = True):
        """Appends a suffix to this element's ID, and optionally to all child IDs as well. There is sually no need to call t
        if self.id: self.id += idsuffix
        if recursive:
```

```
all_token_batches = []

for code in tqdm(code_texts):
```

```
    tokens = ft_model.to_tokens(
        code,
        truncate=True,
        prepend_bos=True
    )

    tokens = tokens[:, :MAX_LENGTH]

    if tokens.shape[1] >= 16:
        all_token_batches.append(tokens.squeeze(0))

print("Number of token sequences:", len(all_token_batches))
print("Example shape:", all_token_batches[0].shape)
```

```
100%|██████████| 3000/3000 [00:07<00:00, 415.38it/s]Number of token sequences: 3000
Example shape: torch.Size([107])
```

```
from torch.utils.data import DataLoader
import torch

pad_token = ft_model.tokenizer.eos_token_id

padded_batches = []

for tokens in all_token_batches:

    tokens = tokens.detach().cpu()

    seq_len = tokens.shape[0]

    if seq_len < MAX_LENGTH:
        pad_len = MAX_LENGTH - seq_len

        padding = torch.full(
            (pad_len,),
            pad_token,
            dtype=tokens.dtype,
            device=tokens.device
        )

        tokens = torch.cat([tokens, padding], dim=0)

    else:
        tokens = tokens[:MAX_LENGTH]

    padded_batches.append(tokens)

token_tensor = torch.stack(padded_batches)

print("Token tensor:", token_tensor.shape)
print("Token tensor device:", token_tensor.device)

ft_loader = DataLoader(
    token_tensor,
    batch_size=FT_BATCH_SIZE,
    shuffle=True,
    drop_last=True
)
```

```
Token tensor: torch.Size([3000, 128])
Token tensor device: cpu
```

```
import torch.nn.functional as F
from tqdm import tqdm

pad_token = ft_model.tokenizer.eos_token_id

optimizer = torch.optim.AdamW(ft_model.parameters(), lr=FT_LR)

ft_model.train()

step = 0
optimizer.zero_grad()

progress = tqdm(total=FT_STEPS)

while step < FT_STEPS:
    for batch_tokens in ft_loader:
        batch_tokens = batch_tokens.to(device)

        logits = ft_model(batch_tokens)
```

```
        logits_for_loss = logits[:, :-1, :]
        targets = batch_tokens[:, 1:]

        loss = F.cross_entropy(
            logits_for_loss.reshape(-1, logits_for_loss.size(-1)),
            targets.reshape(-1),
            ignore_index=pad_token
        )

        loss = loss / GRAD_ACCUM_STEPS
        loss.backward()

        if (step + 1) % GRAD_ACCUM_STEPS == 0:
            torch.nn.utils.clip_grad_norm_(ft_model.parameters(), 1.0)
            optimizer.step()
            optimizer.zero_grad()

        if step % 20 == 0:
            progress.set_description(f"loss={loss.item() * GRAD_ACCUM_STEPS:.4f}")

        step += 1
        progress.update(1)

        if step >= FT_STEPS:
            break

progress.close()
print("Fine-tuning complete.")
```

```
loss=2.5990: 100%|██████████| 500/500 [01:06<00:00,  7.51it/s]Fine-tuning complete.
```

```
ft_model.eval()

prompt = "def add_numbers(a, b):\n"

tokens = ft_model.to_tokens(prompt).to(device)

with torch.no_grad():
    generated = ft_model.generate(
        tokens,
        max_new_tokens=80,
        temperature=0.8,
        top_p=0.95,
        do_sample=True
    )

print(ft_model.to_string(generated[0]))
```

100% 80/80 [00:02<00:00, 33.93it/s]

```
def add_numbers(a, b):
    """Add numbers to the tuple.

    :rtype: tuple
    """
    n = int(a)
    if isinstance(a, tuple):
        n += 1
    elif isinstance(a, d) and a is None:
        d = a
    elif isinstance(a, (tuple,d)) and a is None:
        d =
```

```
SAVE_FT_MODEL_PATH = "pythia160m_code_finetuned.pt"

torch.save(
    {
        "model_state_dict": ft_model.state_dict(),
        "model_name": MODEL_NAME,
        "fine_tune_domain": "python_code",
        "ft_steps": FT_STEPS,
        "ft_lr": FT_LR,
        "max_length": MAX_LENGTH,
    },
    SAVE_FT_MODEL_PATH
)

print("Saved fine-tuned Pythia to:", SAVE_FT_MODEL_PATH)
```

```
Saved fine-tuned Pythia to: pythia160m_code_finetuned.pt
```

```
HOOK_NAME = "blocks.6.hook_resid_post"
```

```
HOOK_NAME = "blocks.6.hook_resid_post"
MAX_ACTIVATIONS = 50_000

ft_model.eval()

ft_all_acts = []
ft_all_token_strs = []

with torch.no_grad():
    for code in tqdm(code_texts):
        if not isinstance(code, str) or len(code.strip()) == 0:
            continue

        tokens = ft_model.to_tokens(
            code,
            truncate=True,
            prepend_bos=True
        ).to(device)

        tokens = tokens[:, :MAX_LENGTH]

        logits, cache = ft_model.run_with_cache(tokens)

        acts = cache[HOOK_NAME]
        acts = acts.reshape(-1, acts.shape[-1])

        token_strs = ft_model.to_str_tokens(tokens[0])

        acts = acts[1:]
        token_strs = token_strs[1:]

        ft_all_acts.append(acts.detach().cpu().float())
        ft_all_token_strs.extend(token_strs)

        total = sum(x.shape[0] for x in ft_all_acts)

        if total >= MAX_ACTIVATIONS:
            break

ft_activations = torch.cat(ft_all_acts, dim=0)[:MAX_ACTIVATIONS]
ft_all_token_strs = ft_all_token_strs[:MAX_ACTIVATIONS]

print("Fine-tuned activation tensor:", ft_activations.shape)
print("Number of token strings:", len(ft_all_token_strs))
print("First 20 tokens:", ft_all_token_strs[:20])
```

```
 15%|█▌        | 437/3000 [00:19<01:54, 22.39it/s]
Fine-tuned activation tensor: torch.Size([50000, 768])
Number of token strings: 50000
First 20 tokens: ['def', ' add', 'ids', 'uffix', '(', 'self', ',', ' ids', 'uffix', ',', ' recursive', ' =', ' True', '):',
```

```
from torch.utils.data import TensorDataset, DataLoader
import torch.nn.functional as F

D_IN = 768
D_SAE = 4096

SAE_BATCH_SIZE = 512
LR = 1e-3
L1_COEFF = 1e-4
EPOCHS = 10

ft_sae = SparseAutoencoder(d_in=D_IN, d_sae=D_SAE).to(device)

ft_train_loader = DataLoader(
    TensorDataset(ft_activations),
    batch_size=SAE_BATCH_SIZE,
    shuffle=True,
    drop_last=True
)

optimizer = torch.optim.Adam(ft_sae.parameters(), lr=LR)

for epoch in range(EPOCHS):
    ft_sae.train()

    total_loss = 0.0
    total_mse = 0.0
```

```
    total_l1 = 0.0
    total_l0 = 0.0

    for batch in tqdm(ft_train_loader, desc=f"FT SAE Epoch {epoch+1}/{EPOCHS}"):
        x = batch[0].to(device)

        reconstruction, features = ft_sae(x)

        mse_loss = F.mse_loss(reconstruction, x)
        l1_loss = features.abs().sum(dim=-1).mean()

        loss = mse_loss + L1_COEFF * l1_loss

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        ft_sae.normalize_decoder()

        with torch.no_grad():
            l0 = (features > 0).float().sum(dim=-1).mean()

        total_loss += loss.item()
        total_mse += mse_loss.item()
        total_l1 += l1_loss.item()
        total_l0 += l0.item()

    n = len(ft_train_loader)

    print(
        f"Epoch {epoch+1} | "
        f"Loss: {total_loss/n:.6f} | "
        f"MSE: {total_mse/n:.6f} | "
        f"L1: {total_l1/n:.6f} | "
        f"Avg L0: {total_l0/n:.2f}"
    )
```

```
FT SAE Epoch 1/10: 100%|██████████| 97/97 [00:00<00:00, 116.48it/s]
Epoch 1 | Loss: 0.263686 | MSE: 0.235995 | L1: 276.911400 | Avg L0: 685.13
FT SAE Epoch 2/10: 100%|██████████| 97/97 [00:00<00:00, 117.12it/s]
Epoch 2 | Loss: 0.072971 | MSE: 0.049556 | L1: 234.156256 | Avg L0: 589.78
FT SAE Epoch 3/10: 100%|██████████| 97/97 [00:00<00:00, 116.84it/s]
Epoch 3 | Loss: 0.053589 | MSE: 0.029936 | L1: 236.526710 | Avg L0: 616.70
FT SAE Epoch 4/10: 100%|██████████| 97/97 [00:00<00:00, 116.80it/s]
Epoch 4 | Loss: 0.045057 | MSE: 0.021741 | L1: 233.159190 | Avg L0: 637.29
FT SAE Epoch 5/10: 100%|██████████| 97/97 [00:00<00:00, 117.05it/s]
Epoch 5 | Loss: 0.040433 | MSE: 0.017582 | L1: 228.501929 | Avg L0: 652.51
FT SAE Epoch 6/10: 100%|██████████| 97/97 [00:00<00:00, 117.57it/s]
Epoch 6 | Loss: 0.037319 | MSE: 0.014949 | L1: 223.692161 | Avg L0: 663.87
FT SAE Epoch 7/10: 100%|██████████| 97/97 [00:00<00:00, 116.91it/s]
Epoch 7 | Loss: 0.035028 | MSE: 0.013152 | L1: 218.766392 | Avg L0: 671.89
FT SAE Epoch 8/10: 100%|██████████| 97/97 [00:00<00:00, 116.49it/s]
Epoch 8 | Loss: 0.033208 | MSE: 0.011789 | L1: 214.197794 | Avg L0: 677.90
FT SAE Epoch 9/10: 100%|██████████| 97/97 [00:00<00:00, 116.25it/s]
Epoch 9 | Loss: 0.032938 | MSE: 0.012033 | L1: 209.053040 | Avg L0: 678.76
FT SAE Epoch 10/10: 100%|██████████| 97/97 [00:00<00:00, 105.63it/s]Epoch 10 | Loss: 0.031038 | MSE: 0.010379 | L1: 206.5858
```

```
ft_sae.eval()

with torch.no_grad():
    x = ft_activations[:2048].to(device)
    reconstruction, features = ft_sae(x)

    ft_mse = F.mse_loss(reconstruction, x).item()
    ft_l0 = (features > 0).float().sum(dim=-1).mean().item()

torch.save(
    {
        "sae_state_dict": ft_sae.state_dict(),
        "model_name": MODEL_NAME,
        "fine_tuned_model_path": SAVE_FT_MODEL_PATH,
        "hook_name": HOOK_NAME,
        "layer": 6,
        "d_in": D_IN,
        "d_sae": D_SAE,
        "max_length": MAX_LENGTH,
        "max_activations": MAX_ACTIVATIONS,
        "l1_coeff": L1_COEFF,
        "epochs": EPOCHS,
        "final_avg_l0": ft_l0,
        "final_mse": ft_mse,
    },
```

```
        "code_finetuned_pythia160m_layer6_sae.pt"
    )

    print("Saved fine-tuned SAE.")
    print("FT SAE MSE:", ft_mse)
    print("FT SAE Avg L0:", ft_l0)
```

```
Saved fine-tuned SAE.
FT SAE MSE: 0.009607473388314247
FT SAE Avg L0: 677.68017578125
```

```
import pandas as pd

@torch.no_grad()
def compare_decoder_directions(sae_a, sae_b, batch_size=512):
    W_a = sae_a.W_dec.detach().cpu()
    W_b = sae_b.W_dec.detach().cpu()

    W_a = F.normalize(W_a, dim=1)
    W_b = F.normalize(W_b, dim=1)

    all_best_scores = []
    all_best_indices = []

    for start in tqdm(range(0, W_a.shape[0], batch_size)):
        end = start + batch_size

        sims = W_a[start:end] @ W_b.T

        best_scores, best_indices = sims.max(dim=1)

        all_best_scores.append(best_scores)
        all_best_indices.append(best_indices)

    best_match_scores = torch.cat(all_best_scores)
    best_match_indices = torch.cat(all_best_indices)

    return best_match_indices, best_match_scores


best_match_indices, best_match_scores = compare_decoder_directions(sae, ft_sae)

print("Mean best cosine similarity:", best_match_scores.mean().item())
print("Median best cosine similarity:", best_match_scores.median().item())
print("Min:", best_match_scores.min().item())
print("Max:", best_match_scores.max().item())
```

```
100%|██████████| 8/8 [00:00<00:00, 23.01it/s]Mean best cosine similarity: 0.14298202097415924
Median best cosine similarity: 0.13249626755714417
Min: 0.10420390963554382
Max: 0.7346885800361633
```

```
comparison_df = pd.DataFrame({
    "baseline_feature": list(range(D_SAE)),
    "matched_ft_feature": best_match_indices.numpy(),
    "decoder_cosine_similarity": best_match_scores.numpy(),
})

comparison_df["status"] = comparison_df["decoder_cosine_similarity"].apply(
    lambda x: "preserved" if x > 0.80 else ("shifted" if x > 0.50 else "changed")
)

comparison_df.to_csv("sae_feature_decoder_comparison.csv", index=False)

print(comparison_df["status"].value_counts())
comparison_df.head()
```

```
status
changed    4084
shifted      12
Name: count, dtype: int64
```

	baseline_feature	matched_ft_feature	decoder_cosine_similarity	status
0	0	3200	0.128010	changed
1	1	706	0.121333	changed
2	2	3200	0.118381	changed
3	3	3262	0.126683	changed
4	4	338	0.141986	changed

Next steps: Generate code with comparison_df New interactive sheet

```
comparison_df = pd.DataFrame({
    "baseline_feature": list(range(D_SAE)),
    "matched_ft_feature": best_match_indices.numpy(),
    "decoder_cosine_similarity": best_match_scores.numpy(),
})

comparison_df["status"] = comparison_df["decoder_cosine_similarity"].apply(
    lambda x: "preserved" if x > 0.80 else ("shifted" if x > 0.50 else "changed")
)

comparison_df.to_csv("sae_feature_decoder_comparison.csv", index=False)

print(comparison_df["status"].value_counts())
comparison_df.head()
```

```
status
changed    4084
shifted      12
Name: count, dtype: int64
```

	baseline_feature	matched_ft_feature	decoder_cosine_similarity	status
0	0	3200	0.128010	changed
1	1	706	0.121333	changed
2	2	3200	0.118381	changed
3	3	3262	0.126683	changed
4	4	338	0.141986	changed

Next steps: Generate code with comparison_df New interactive sheet

```
comparison_df = pd.DataFrame({
    "baseline_feature": list(range(D_SAE)),
    "matched_ft_feature": best_match_indices.numpy(),
    "decoder_cosine_similarity": best_match_scores.numpy(),
})

comparison_df["status"] = comparison_df["decoder_cosine_similarity"].apply(
    lambda x: "preserved" if x > 0.80 else ("shifted" if x > 0.50 else "changed")
)

comparison_df.to_csv("sae_feature_decoder_comparison.csv", index=False)

print(comparison_df["status"].value_counts())
comparison_df.head()
```

```
status
changed    4084
shifted      12
Name: count, dtype: int64
```

	baseline_feature	matched_ft_feature	decoder_cosine_similarity	status
0	0	3200	0.128010	changed
1	1	706	0.121333	changed
2	2	3200	0.118381	changed
3	3	3262	0.126683	changed
4	4	338	0.141986	changed

Next steps: Generate code with comparison_df New interactive sheet

```
comparison_df.sort_values("decoder_cosine_similarity", ascending=False).head(20)
```

	baseline_feature	matched_ft_feature	decoder_cosine_similarity	status
137	137	484	0.734689	shifted
3889	3889	2184	0.719999	shifted
716	716	484	0.677752	shifted
201	201	276	0.587130	shifted
1842	1842	1341	0.567703	shifted
907	907	304	0.543653	shifted
293	293	3598	0.523310	shifted
1934	1934	753	0.522680	shifted
1956	1956	2095	0.522364	shifted
2836	2836	1464	0.517482	shifted
887	887	535	0.512469	shifted
301	301	4018	0.508364	shifted
3383	3383	2184	0.496616	changed
2545	2545	1763	0.495081	changed
1506	1506	2131	0.479276	changed
2617	2617	4053	0.476383	changed
1629	1629	1212	0.471651	changed
1875	1875	1470	0.462660	changed
3729	3729	3571	0.460631	changed
2695	2695	1706	0.458688	changed

```
comparison_df.sort_values("decoder_cosine_similarity").head(20)
```

	baseline_feature	matched_ft_feature	decoder_cosine_similarity	status
2526	2526	2728	0.104204	changed
1942	1942	1624	0.105560	changed
2923	2923	2922	0.105774	changed
3337	3337	1613	0.106735	changed
2914	2914	3658	0.107202	changed
608	608	67	0.107503	changed
1716	1716	3553	0.107755	changed
3574	3574	1574	0.108236	changed
3816	3816	4026	0.108246	changed
1199	1199	1059	0.108277	changed
2500	2500	3301	0.108533	changed
1899	1899	2257	0.108782	changed
2044	2044	3790	0.109099	changed
1065	1065	3013	0.109109	changed
1548	1548	3952	0.109543	changed
3231	3231	1707	0.109638	changed
3330	3330	2145	0.109736	changed
2155	2155	3728	0.109770	changed
3804	3804	318	0.109881	changed
2573	2573	1590	0.110066	changed

```
baseline_feature = 500

matched_ft_feature = best_match_indices[baseline_feature].item()
sim = best_match_scores[baseline_feature].item()

print("Baseline feature:", baseline_feature)
```

```
print("Matched fine-tuned feature:", matched_ft_feature)
print("Decoder cosine similarity:", sim)
```

```
Baseline feature: 500
Matched fine-tuned feature: 1717
Decoder cosine similarity: 0.132863387465477
```

```
print("Baseline feature top tokens")
base_top = get_top_activating_tokens(
    sae=sae,
    activations=activations,
    token_strs=all_token_strs,
    feature_idx=baseline_feature,
    top_k=20
)

for row in base_top:
    print(f"{row['activation']:.4f} | {repr(row['token'])}")
```

```
Baseline feature top tokens
1.3053 | ' nanot'
1.2573 | ' quint'
1.2177 | ' on'
1.1894 | ' bee'
1.1161 | 'oc'
1.0975 | ' bee'
1.0848 | ' pro'
1.0800 | ' plane'
1.0750 | 'order'
1.0728 | 'oc'
1.0641 | 'Po'
1.0554 | 'oc'
1.0390 | '}^\\'
1.0336 | ' stick'
1.0284 | 'ithe'
1.0045 | ' the'
0.9907 | ' party'
0.9890 | ' visit'
0.9865 | 'oc'
0.9852 | '14'
```

```
print("Fine-tuned matched feature top tokens")
ft_top = get_top_activating_tokens(
    sae=ft_sae,
    activations=ft_activations,
    token_strs=ft_all_token_strs,
    feature_idx=matched_ft_feature,
    top_k=20
)

for row in ft_top:
    print(f"{row['activation']:.4f} | {repr(row['token'])}")
```

```
Fine-tuned matched feature top tokens
0.0000 | 'ends'
0.0000 | ' recursive'
0.0000 | '('
0.0000 | '"""'
0.0000 | ' True'
0.0000 | 'uffix'
0.0000 | ' a'
0.0000 | ' add'
0.0000 | ' ='
0.0000 | 'self'
0.0000 | '        '
0.0000 | '):'
0.0000 | ' ids'
0.0000 | 'App'
0.0000 | 'uffix'
0.0000 | ','
0.0000 | 'def'
0.0000 | 'ids'
0.0000 | ','
0.0000 | '\n'
```

```
ft_to_base_indices, ft_to_base_scores = compare_decoder_directions(ft_sae, sae)

ft_new_df = pd.DataFrame({
    "ft_feature": list(range(D_SAE)),
    "matched_baseline_feature": ft_to_base_indices.numpy(),
    "decoder_cosine_similarity_to_baseline": ft_to_base_scores.numpy(),
})

ft_new_df["status"] = ft_new_df["decoder_cosine_similarity_to_baseline"].apply(
    lambda x: "preserved_from_baseline" if x > 0.80 else ("shifted_from_baseline" if x > 0.50 else "possibly_new")
```

```
)

ft_new_df.to_csv("fine_tuned_possible_new_features.csv", index=False)

ft_new_df.sort_values("decoder_cosine_similarity_to_baseline").head(20)
```

```
100%|██████████| 8/8 [00:00<00:00, 25.86it/s]
```

	ft_feature	matched_baseline_feature	decoder_cosine_similarity_to_baseline	status
3447	3447	3931	0.102445	possibly_new
1748	1748	137	0.104901	possibly_new
487	487	3253	0.106908	possibly_new
3960	3960	2842	0.107153	possibly_new
3963	3963	480	0.107528	possibly_new
4089	4089	1889	0.107610	possibly_new
1356	1356	3742	0.107972	possibly_new
1962	1962	2993	0.108138	possibly_new
3462	3462	1892	0.108154	possibly_new
717	717	651	0.108544	possibly_new
1358	1358	279	0.108755	possibly_new
3493	3493	1591	0.109215	possibly_new
3188	3188	2100	0.109320	possibly_new
3969	3969	2236	0.109336	possibly_new
303	303	234	0.109349	possibly_new
1331	1331	2966	0.109360	possibly_new
4063	4063	921	0.109607	possibly_new
695	695	524	0.109688	possibly_new
178	178	513	0.109735	possibly_new
1128	1128	2188	0.109820	possibly_new

```
possible_new_features = ft_new_df.sort_values(
    "decoder_cosine_similarity_to_baseline"
).head(10)["ft_feature"].tolist()

for feat in possible_new_features:
    print("\n" + "#" * 100)
    print("Fine-tuned feature:", feat)
    print("Similarity to closest baseline feature:", ft_to_base_scores[feat].item())

    top_tokens = get_top_activating_tokens(
        sae=ft_sae,
        activations=ft_activations,
        token_strs=ft_all_token_strs,
        feature_idx=feat,
        top_k=15
    )

    for row in top_tokens:
        print(f"{row['activation']:.4f} | {repr(row['token'])}")
```

```
####################################################################################################
Fine-tuned feature: 3447
Similarity to closest baseline feature: 0.10244455933570862
0.3506 | ' with'
0.1841 | ' use'
0.1680 | ' using'
0.1653 | '�'
0.1546 | ' do'
0.1411 | ' +='
0.1372 | ' to'
0.1364 | '�'
0.1346 | 'Cor'
0.1284 | '.'
0.1268 | 'Cor'
0.1258 | 'Cor'
0.1257 | 'For'
0.1230 | '3'
0.1215 | 'Cor'

####################################################################################################
```

```
Fine-tuned feature: 1748
Similarity to closest baseline feature: 0.10490149259567261
1.5586 | '\n'
1.5338 | '\n'
1.4893 | '\n'
1.4744 | '\n'
1.4727 | '\n'
1.4280 | '\n'
1.3107 | '\n'
1.2960 | '\n'
1.2839 | '\n'
1.2741 | '\n'
1.2685 | '\n'
1.2657 | '\n'
1.2603 | '\n'
1.2517 | '\n'
1.2088 | '\n'

####################################################################################################
Fine-tuned feature: 487
Similarity to closest baseline feature: 0.10690803825855255
0.2685 | ' dedicated'
0.2562 | '000'
0.2507 | 'absolute'
0.2497 | ' corrected'
0.2162 | 'absolute'
0.1997 | 'absolute'
0.1992 | 'absolute'
0.1881 | ' deprecated'
0.1881 | ' deprecated'
0.1881 | ' deprecated'
0.1881 | ' deprecated'
0.1881 | ' deprecated'
0.1626 | ' promised'
0.1617 | ' contain'
0.1311 | 'ited'
```

```
pre_code_model = HookedTransformer.from_pretrained(
    MODEL_NAME,
    device=device
)

pre_code_model.eval()

for p in pre_code_model.parameters():
    p.requires_grad = False

print("Loaded fresh pretrained Pythia for code activations")
```

Loading weights: 100% 148/148 [00:00<00:00, 353.03it/s]

```
Loaded pretrained model EleutherAI/pythia-160m into HookedTransformer
Loaded fresh pretrained Pythia for code activations
```

```
HOOK_NAME = "blocks.6.hook_resid_post"
MAX_ACTIVATIONS = 50_000

pre_code_all_acts = []
pre_code_all_token_strs = []

with torch.no_grad():
    for code in tqdm(code_texts):
        if not isinstance(code, str) or len(code.strip()) == 0:
            continue

        tokens = pre_code_model.to_tokens(
            code,
            truncate=True,
            prepend_bos=True
        ).to(device)

        tokens = tokens[:, :MAX_LENGTH]

        logits, cache = pre_code_model.run_with_cache(tokens)

        acts = cache[HOOK_NAME]
        acts = acts.reshape(-1, acts.shape[-1])

        token_strs = pre_code_model.to_str_tokens(tokens[0])


        acts = acts[1:]
        token_strs = token_strs[1:]

        pre_code_all_acts.append(acts.detach().cpu().float())
```

```
        pre_code_all_token_strs.extend(token_strs)

        total = sum(x.shape[0] for x in pre_code_all_acts)

        if total >= MAX_ACTIVATIONS:
            break

pre_code_activations = torch.cat(pre_code_all_acts, dim=0)[:MAX_ACTIVATIONS]
pre_code_all_token_strs = pre_code_all_token_strs[:MAX_ACTIVATIONS]

print("Pretrained-code activation tensor:", pre_code_activations.shape)
print("Number of token strings:", len(pre_code_all_token_strs))
print("First 20 tokens:", pre_code_all_token_strs[:20])
```

```
 15%|█▍        | 437/3000 [00:22<02:14, 19.10it/s]
Pretrained-code activation tensor: torch.Size([50000, 768])
Number of token strings: 50000
First 20 tokens: ['def', ' add', 'ids', 'uffix', '(', 'self', ',', ' ids', 'uffix', ',', ' recursive', ' =', ' True', '):',
```

```
D_IN = 768
D_SAE = 4096

SAE_BATCH_SIZE = 512
LR = 1e-3
L1_COEFF = 1e-4
EPOCHS = 10

pre_code_sae = SparseAutoencoder(d_in=D_IN, d_sae=D_SAE).to(device)

pre_code_train_loader = DataLoader(
    TensorDataset(pre_code_activations),
    batch_size=SAE_BATCH_SIZE,
    shuffle=True,
    drop_last=True
)

optimizer = torch.optim.Adam(pre_code_sae.parameters(), lr=LR)

for epoch in range(EPOCHS):
    pre_code_sae.train()

    total_loss = 0.0
    total_mse = 0.0
    total_l1 = 0.0
    total_l0 = 0.0

    for batch in tqdm(pre_code_train_loader, desc=f"Pre-code SAE Epoch {epoch+1}/{EPOCHS}"):
        x = batch[0].to(device)

        reconstruction, features = pre_code_sae(x)

        mse_loss = F.mse_loss(reconstruction, x)
        l1_loss = features.abs().sum(dim=-1).mean()

        loss = mse_loss + L1_COEFF * l1_loss

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        pre_code_sae.normalize_decoder()

        with torch.no_grad():
            l0 = (features > 0).float().sum(dim=-1).mean()

        total_loss += loss.item()
        total_mse += mse_loss.item()
        total_l1 += l1_loss.item()
        total_l0 += l0.item()

    n = len(pre_code_train_loader)

    print(
        f"Epoch {epoch+1} | "
        f"Loss: {total_loss/n:.6f} | "
        f"MSE: {total_mse/n:.6f} | "
        f"L1: {total_l1/n:.6f} | "
        f"Avg L0: {total_l0/n:.2f}"
    )
```

```
Pre-code SAE Epoch 1/10: 100%|██████████| 97/97 [00:01<00:00, 65.35it/s]
Epoch 1 | Loss: 0.308280 | MSE: 0.282585 | L1: 256.946919 | Avg L0: 634.79
Pre-code SAE Epoch 2/10: 100%|██████████| 97/97 [00:00<00:00, 99.70it/s]
Epoch 2 | Loss: 0.075034 | MSE: 0.053256 | L1: 217.780358 | Avg L0: 529.98
Pre-code SAE Epoch 3/10: 100%|██████████| 97/97 [00:00<00:00, 103.12it/s]
Epoch 3 | Loss: 0.055524 | MSE: 0.032946 | L1: 225.778556 | Avg L0: 559.17
Pre-code SAE Epoch 4/10: 100%|██████████| 97/97 [00:01<00:00, 96.45it/s]
Epoch 4 | Loss: 0.047036 | MSE: 0.024298 | L1: 227.381989 | Avg L0: 580.01
Pre-code SAE Epoch 5/10: 100%|██████████| 97/97 [00:01<00:00, 88.15it/s]
Epoch 5 | Loss: 0.042222 | MSE: 0.019718 | L1: 225.049395 | Avg L0: 595.00
Pre-code SAE Epoch 6/10: 100%|██████████| 97/97 [00:00<00:00, 107.32it/s]
Epoch 6 | Loss: 0.039249 | MSE: 0.017022 | L1: 222.273505 | Avg L0: 607.87
Pre-code SAE Epoch 7/10: 100%|██████████| 97/97 [00:00<00:00, 118.44it/s]
Epoch 7 | Loss: 0.036582 | MSE: 0.014640 | L1: 219.412955 | Avg L0: 617.37
Pre-code SAE Epoch 8/10: 100%|██████████| 97/97 [00:00<00:00, 118.20it/s]
Epoch 8 | Loss: 0.034862 | MSE: 0.013306 | L1: 215.558107 | Avg L0: 623.40
Pre-code SAE Epoch 9/10: 100%|██████████| 97/97 [00:00<00:00, 117.06it/s]
Epoch 9 | Loss: 0.033443 | MSE: 0.012270 | L1: 211.729849 | Avg L0: 628.17
Pre-code SAE Epoch 10/10: 100%|██████████| 97/97 [00:00<00:00, 117.04it/s]Epoch 10 | Loss: 0.033072 | MSE: 0.012276 | L1: 20
```

```
pre_code_sae.eval()

with torch.no_grad():
    x = pre_code_activations[:2048].to(device)
    reconstruction, features = pre_code_sae(x)

    pre_code_mse = F.mse_loss(reconstruction, x).item()
    pre_code_l0 = (features > 0).float().sum(dim=-1).mean().item()

torch.save(
    {
        "sae_state_dict": pre_code_sae.state_dict(),
        "model_name": MODEL_NAME,
        "hook_name": HOOK_NAME,
        "layer": 6,
        "d_in": D_IN,
        "d_sae": D_SAE,
        "max_length": MAX_LENGTH,
        "max_activations": MAX_ACTIVATIONS,
        "l1_coeff": L1_COEFF,
        "epochs": EPOCHS,
        "final_avg_l0": pre_code_l0,
        "final_mse": pre_code_mse,
        "training_data": "python_code",
        "base_model": "pretrained_pythia160m"
    },
    "pretrained_pythia160m_code_layer6_sae.pt"
)

print("Saved pretrained-code SAE.")
print("Pre-code SAE MSE:", pre_code_mse)
print("Pre-code SAE Avg L0:", pre_code_l0)
```

```
Saved pretrained-code SAE.
Pre-code SAE MSE: 0.01198769360780716
Pre-code SAE Avg L0: 628.3056640625
```

```
best_match_indices, best_match_scores = compare_decoder_directions(
    pre_code_sae,
    ft_sae
)

print("Mean best cosine similarity:", best_match_scores.mean().item())
print("Median best cosine similarity:", best_match_scores.median().item())
print("Min:", best_match_scores.min().item())
print("Max:", best_match_scores.max().item())
```

```
100%|██████████| 8/8 [00:00<00:00, 24.05it/s]Mean best cosine similarity: 0.15625758469104767
Median best cosine similarity: 0.13364242017269135
Min: 0.10556469857692719
Max: 0.7367433309555054
```

```
comparison_df = pd.DataFrame({
    "pretrained_code_feature": list(range(D_SAE)),
    "matched_finetuned_code_feature": best_match_indices.numpy(),
    "decoder_cosine_similarity": best_match_scores.numpy(),
})

comparison_df["status"] = comparison_df["decoder_cosine_similarity"].apply(
    lambda x: "preserved" if x > 0.80 else ("shifted" if x > 0.50 else "changed")
```

```
)

comparison_df.to_csv("proper_code_sae_feature_decoder_comparison.csv", index=False)

print(comparison_df["status"].value_counts())
comparison_df.sort_values("decoder_cosine_similarity", ascending=False).head(20)
```

```
status
changed    4058
shifted      38
Name: count, dtype: int64
```

	pretrained_code_feature	matched_finetuned_code_feature	decoder_cosine_similarity	status
494	494	1763	0.736743	shifted
1959	1959	2579	0.716393	shifted
433	433	3090	0.658942	shifted
1287	1287	2671	0.652647	shifted
838	838	2876	0.639546	shifted
2583	2583	3654	0.634729	shifted
419	419	2398	0.623007	shifted
841	841	2314	0.621928	shifted
3502	3502	548	0.620208	shifted
1987	1987	1633	0.616685	shifted
1115	1115	3118	0.601689	shifted
3595	3595	2061	0.593636	shifted
2544	2544	2184	0.591755	shifted
1621	1621	813	0.587619	shifted
3330	3330	974	0.585188	shifted
3673	3673	2080	0.584436	shifted
1288	1288	1716	0.584341	shifted
3920	3920	3335	0.582997	shifted
3685	3685	447	0.572484	shifted
4080	4080	3220	0.570344	shifted

```
ft_stats = get_feature_stats(ft_sae, ft_activations)

print("FT dead features:", (ft_stats["firing_rate"] == 0).sum().item())
print("FT features firing > 1%:", (ft_stats["firing_rate"] > 0.01).sum().item())
print("FT features firing > 10%:", (ft_stats["firing_rate"] > 0.10).sum().item())
```

```
100%|██████████| 49/49 [00:02<00:00, 20.62it/s]FT dead features: 348
FT features firing > 1%: 1568
FT features firing > 10%: 1097
```

```
ft_to_base_indices, ft_to_base_scores = compare_decoder_directions(ft_sae, pre_code_sae)

ft_new_df = pd.DataFrame({
    "ft_feature": list(range(D_SAE)),
    "matched_pre_code_feature": ft_to_base_indices.numpy(),
    "decoder_cosine_similarity_to_pre_code": ft_to_base_scores.numpy(),
    "firing_rate": ft_stats["firing_rate"].numpy(),
    "max_activation": ft_stats["max_activation"].numpy(),
})

candidate_new_df = ft_new_df[
    (ft_new_df["decoder_cosine_similarity_to_pre_code"] < 0.30) &
    (ft_new_df["firing_rate"] > 0.005) &
    (ft_new_df["max_activation"] > 0.5)
].sort_values("decoder_cosine_similarity_to_pre_code")

candidate_new_df.head(20)
```

```
100%|██████████| 8/8 [00:00<00:00, 13.57it/s]
```

	ft_feature	matched_pre_code_feature	decoder_cosine_similarity_to_pre_code	firing_rate	max_activation
2434	2434	3710	0.109645	0.41594	1.587685
1387	1387	1533	0.109650	0.24356	5.349433
1494	1494	100	0.109869	0.06968	0.623343
1090	1090	1852	0.110178	0.01634	0.620171
4061	4061	817	0.111344	0.58924	1.013143
2092	2092	2762	0.112307	0.00550	0.683496
334	334	3633	0.112725	0.07828	0.581172
1952	1952	1733	0.112789	0.57536	1.006076
3753	3753	3215	0.113103	0.40360	0.789442
926	926	922	0.113475	0.06188	0.544589
1413	1413	3418	0.113945	0.01188	0.570747
870	870	3448	0.114138	0.01058	0.520841
1647	1647	3751	0.114678	0.42070	2.971415
1530	1530	2899	0.114805	0.00698	0.631127
3651	3651	3167	0.114961	0.01294	0.587213
1811	1811	3849	0.115244	0.46106	0.721878
2904	2904	699	0.115403	0.56924	1.051047
4000	4000	3877	0.115439	0.49676	0.767936
792	792	1724	0.115564	0.22570	0.782764
255	255	2907	0.115823	0.32038	0.704893

Next steps: Generate code with candidate_new_df New interactive sheet

```
candidate_features = candidate_new_df.head(10)["ft_feature"].tolist()

for feat in candidate_features:
    print("\n" + "#" * 100)
    print("Fine-tuned candidate-new feature:", feat)
    print("Similarity to closest pretrained-code feature:", ft_to_base_scores[feat].item())
    print("Firing rate:", ft_stats["firing_rate"][feat].item())
    print("Max activation:", ft_stats["max_activation"][feat].item())

    top_tokens = get_top_activating_tokens(
        sae=ft_sae,
        activations=ft_activations,
        token_strs=ft_all_token_strs,
        feature_idx=feat,
        top_k=15
    )

    for row in top_tokens:
        print(f"{row['activation']:.4f} | {repr(row['token'])}")
```

```
0.7729 | 'CON'
0.7696 | 'notation'
0.7609 | 'empty'
0.7536 | 'notation'
0.7534 | 'Con'
0.7100 | 'QU'
0.7092 | 'CON'
0.6860 | 'equ'
0.6855 | ' )'
0.6404 | 'ENC'
0.6100 | 'Abstract'
0.6082 | 'E'
0.6064 | 'E'

####################################################################################################
Fine-tuned candidate-new feature: 926
Similarity to closest pretrained-code feature: 0.11347457766532898
Firing rate: 0.061879999935626984
Max activation: 0.5445892214775085
0.5446 | ' see'
0.5263 | 'select'
0.5044 | ' see'
0.5018 | ' an'
0.4863 | ' deep'
0.4839 | 'See'
0.4707 | 'See'
0.4705 | 'See'
0.4619 | ' dtype'
0.4615 | ' derived'
0.4565 | 'See'
0.4444 | 'utf'
0.4377 | 'See'
0.4175 | ' upon'
0.4103 | 'ub'
```

```python
import pandas as pd
import numpy as np
import torch
import torch.nn.functional as F

summary = {
    "pre_code_mse": pre_code_mse,
    "pre_code_avg_l0": pre_code_l0,
    "ft_code_mse": ft_mse,
    "ft_code_avg_l0": ft_l0,
    "mean_decoder_best_cosine": best_match_scores.mean().item(),
    "median_decoder_best_cosine": best_match_scores.median().item(),
    "min_decoder_best_cosine": best_match_scores.min().item(),
    "max_decoder_best_cosine": best_match_scores.max().item(),
    "num_features": D_SAE,
    "num_activation_vectors": MAX_ACTIVATIONS,
    "layer": 6,
    "hook_name": HOOK_NAME,
}

summary_df = pd.DataFrame([summary])
summary_df.to_csv("evaluation_quantitative_summary.csv", index=False)

summary_df
```

	pre_code_mse	pre_code_avg_l0	ft_code_mse	ft_code_avg_l0	mean_decoder_best_cosine	median_decoder_best_cosine	min_dec
0	0.011988	628.305664	0.009607	677.680176	0.156258	0.133642	

```python
def classify_feature(sim):
    if sim >= 0.50:
        return "relatively_preserved"
    elif sim >= 0.25:
        return "partially_shifted"
    else:
        return "strongly_changed_or_unmatched"

comparison_df["prototype_status"] = comparison_df["decoder_cosine_similarity"].apply(classify_feature)

status_counts = comparison_df["prototype_status"].value_counts().reset_index()
status_counts.columns = ["status", "count"]
status_counts["percentage"] = 100 * status_counts["count"] / D_SAE

status_counts.to_csv("feature_status_counts.csv", index=False)
status_counts
```

	status	count	percentage
0	strongly_changed_or_unmatched	3790	92.529297
1	partially_shifted	268	6.542969
2	relatively_preserved	38	0.927734

Next steps: Generate code with status_counts New interactive sheet

```
most_preserved = comparison_df.sort_values(
    "decoder_cosine_similarity", ascending=False
).head(30)

most_changed = comparison_df.sort_values(
    "decoder_cosine_similarity", ascending=True
).head(30)

most_preserved.to_csv("most_preserved_features.csv", index=False)
most_changed.to_csv("most_changed_features.csv", index=False)

display(most_preserved.head(10))
display(most_changed.head(10))
```

	pretrained_code_feature	matched_finetuned_code_feature	decoder_cosine_similarity	status	prototype_status
494	494	1763	0.736743	shifted	relatively_preserved
1959	1959	2579	0.716393	shifted	relatively_preserved
433	433	3090	0.658942	shifted	relatively_preserved
1287	1287	2671	0.652647	shifted	relatively_preserved
838	838	2876	0.639546	shifted	relatively_preserved
2583	2583	3654	0.634729	shifted	relatively_preserved
419	419	2398	0.623007	shifted	relatively_preserved
841	841	2314	0.621928	shifted	relatively_preserved
3502	3502	548	0.620208	shifted	relatively_preserved
1987	1987	1633	0.616685	shifted	relatively_preserved

	pretrained_code_feature	matched_finetuned_code_feature	decoder_cosine_similarity	status	prototype_stat
1144	1144	724	0.105565	changed	strongly_changed_or_unmatcl
2777	2777	932	0.105978	changed	strongly_changed_or_unmatcl
1780	1780	3486	0.106974	changed	strongly_changed_or_unmatcl
2982	2982	231	0.107162	changed	strongly_changed_or_unmatcl
3727	3727	3582	0.107220	changed	strongly_changed_or_unmatcl
3964	3964	523	0.107421	changed	strongly_changed_or_unmatcl
1561	1561	602	0.107607	changed	strongly_changed_or_unmatcl
824	824	3516	0.107619	changed	strongly_changed_or_unmatcl
2986	2986	1075	0.107673	changed	strongly_changed_or_unmatcl
3895	3895	1679	0.107818	changed	strongly_changed_or_unmatcl

```
def top_token_list(sae_model, acts, token_strs, feature_idx, top_k=10):
    rows = get_top_activating_tokens(
        sae=sae_model,
        activations=acts,
        token_strs=token_strs,
        feature_idx=feature_idx,
        top_k=top_k
    )
    return [r["token"] for r in rows], [r["activation"] for r in rows]

def summarize_feature_pair(pre_feat_idx, top_k=10):
    ft_feat_idx = int(best_match_indices[pre_feat_idx].item())
    sim = float(best_match_scores[pre_feat_idx].item())

    pre_tokens, pre_scores = top_token_list(
        pre_code_sae,
        pre_code_activations,
        pre_code_all_token_strs,
        pre_feat_idx,
```

```
        top_k=top_k
    )

    ft_tokens, ft_scores = top_token_list(
        ft_sae,
        ft_activations,
        ft_all_token_strs,
        ft_feat_idx,
        top_k=top_k
    )

    return {
        "pretrained_code_feature": pre_feat_idx,
        "matched_finetuned_feature": ft_feat_idx,
        "decoder_cosine_similarity": sim,
        "pre_top_tokens": ", ".join([repr(t) for t in pre_tokens]),
        "ft_top_tokens": ", ".join([repr(t) for t in ft_tokens]),
    }


example_feature_ids = (
    most_preserved["pretrained_code_feature"].head(10).tolist()
    + most_changed["pretrained_code_feature"].head(10).tolist()
)

feature_pair_summaries = [summarize_feature_pair(i, top_k=10) for i in example_feature_ids]

feature_pair_df = pd.DataFrame(feature_pair_summaries)
feature_pair_df.to_csv("feature_pair_top_token_summary.csv", index=False)

feature_pair_df
```

	pretrained_code_feature	matched_finetuned_feature	decoder_cosine_similarity	pre_top_tokens	ft_top_tokens
0	494	1763	0.736743	'或', ' or', '或', '或', '或', '或', 'or', ' or', '...	' or', ' or', ' or', 'or', '或', '或', '或', '或',...
1	1959	2579	0.716393	'\n', '\n', '\n', '\n', '\n', '\n', '\n', '\n'...	'\n', '\n', '\n', '\n', '\n', '\n', '\n', '\n'...
2	433	3090	0.658942	' ', ' ', ' ...	' ', ' ', ' '...
3	1287	2671	0.652647	' ', ' ', ' '...	' ', ' ', ' ...
4	838	2876	0.639546	' =', ' =', ' =', ' =', ' =', ' =', ' =', ' ='...	' =', ' =', ' =', ' =', ' =', ' =', ' =', ' ='...
5	2583	3654	0.634729	' want', ' want', ' want', ' want', ' want', '...	' want', ' want', ' want', ' want', ' want', '...
6	419	2398	0.623007	'(', '(', '(', '(', '(', '(', '(', '(', '(', '('	'(', '(', '(', '(', '(', '(', '(', '(', '(', '('
7	841	2314	0.621928	'\n\n', '\n\n', '\n\n', '\n\n', '\n\n', '\n\n'...	'\n\n', '\n\n', '\n\n', '\n\n', '\n\n', '\n\n'...
8	3502	548	0.620208	' ', ' ', ' ', ' '...	' ', ' ', ' ', ' ', ' ', ' '...
9	1987	1633	0.616685	',', ',', ',', ',', ',', ',', ',', ',', ',', '...	',', ',', ',', ',', ',', ',', ',', ',', ',', '...
10	1144	724	0.105565	' from', ' all', ' into', ' from', '其', ' to',...	' lemma', ' by', 'set', 'path', 's', 'uffix', ...
11	2777	932	0.105978	'1999', ' key', '\n', ' to', ' especially', 't...	'\n', '\n', '\n', '\n', '\n', '\n', '\n', '\n'...
12	1780	3486	0.106974	'x', '.', '.', '3', '/', ' ids', 'uffix', 'uff...	' as', ' as', ' as', ' as', ' as', ' as', ' as...
				'\n', 'AST', '	[illegible]

Next steps: Generate code with feature_pair_df New interactive sheet

```
candidate_new_df_strict = ft_new_df[
    (ft_new_df["decoder_cosine_similarity_to_pre_code"] < 0.30) &
    (ft_new_df["firing_rate"] > 0.002) &
    (ft_new_df["firing_rate"] < 0.10) &
    (ft_new_df["max_activation"] > 0.5)
].sort_values("decoder_cosine_similarity_to_pre_code")

candidate_new_df_strict.to_csv("candidate_new_code_features_strict.csv", index=False)

candidate_new_df_strict.head(30)
```

	ft_feature	matched_pre_code_feature	decoder_cosine_similarity_to_pre_code	firing_rate	max_activation
1494	1494	100	0.109869	0.06968	0.623343
1090	1090	1852	0.110178	0.01634	0.620171
3932	3932	6	0.110797	0.00298	0.563424
3668	3668	1690	0.111028	0.00404	0.508663
2092	2092	2762	0.112307	0.00550	0.683496
334	334	3633	0.112725	0.07828	0.581172
926	926	922	0.113475	0.06188	0.544589
1413	1413	3418	0.113945	0.01188	0.570747
870	870	3448	0.114138	0.01058	0.520841
3027	3027	1755	0.114675	0.00360	0.932133
1530	1530	2899	0.114805	0.00698	0.631127
3651	3651	3167	0.114961	0.01294	0.587213
3328	3328	1450	0.115083	0.00348	0.757453
234	234	2975	0.115840	0.01002	0.576104
3160	3160	902	0.115952	0.02320	0.526466
1613	1613	3015	0.116059	0.00414	0.522785
203	203	523	0.116109	0.01600	0.896010
982	982	2923	0.116603	0.02028	0.569638
173	173	3951	0.116745	0.02588	0.525550
3942	3942	2787	0.116864	0.02370	0.596298
2451	2451	653	0.117104	0.00690	0.516621
2958	2958	3694	0.117132	0.00276	0.519053
3534	3534	2561	0.117363	0.05172	1.641512
2325	2325	2440	0.117470	0.01536	1.516384
3049	3049	3901	0.117502	0.03922	7.669583
4082	4082	2117	0.117560	0.01570	0.520767
3247	3247	3240	0.117603	0.00454	0.537451
3355	3355	1624	0.117631	0.01434	3.669787
420	420	2206	0.117862	0.01704	0.557163
892	892	3055	0.117888	0.01018	1.101979

Next steps: Generate code with `candidate_new_df_strict` New interactive sheet

```
def show_token_context(token_strs, center_idx, window=12):
    start = max(0, center_idx - window)
    end = min(len(token_strs), center_idx + window + 1)

    context = token_strs[start:end].copy()
    center_local_idx = center_idx - start
    context[center_local_idx] = f"[[{context[center_local_idx]}]]"

    return "".join(context)

def get_top_contexts(sae_model, acts, token_strs, feature_idx, top_k=5, window=12):
    rows = get_top_activating_tokens(
        sae=sae_model,
        activations=acts,
        token_strs=token_strs,
        feature_idx=feature_idx,
        top_k=top_k
    )

    contexts = []
    for r in rows:
        contexts.append({
            "activation": r["activation"],
            "token": r["token"],
            "context": show_token_context(token_strs, r["index"], window=window)
        })
```

```
    return contexts
```

```
candidate_features_to_interpret = [2092, 3960, 4089, 3462, 1090]

qual_rows = []

for feat in candidate_features_to_interpret:
    contexts = get_top_contexts(
        sae_model=ft_sae,
        acts=ft_activations,
        token_strs=ft_all_token_strs,
        feature_idx=feat,
        top_k=5,
        window=12
    )

    for c in contexts:
        qual_rows.append({
            "ft_feature": feat,
            "activation": c["activation"],
            "token": c["token"],
            "context": c["context"]
        })

qualitative_context_df = pd.DataFrame(qual_rows)
qualitative_context_df.to_csv("qualitative_context_examples.csv", index=False)

qualitative_context_df
```

	ft_feature	activation	token	context
0	2092	0.683496	:	_kwargs: see config.Config for more details\n ...
1	2092	0.656572	:	����的��单\n \n [[:]]param uid: 用户的ID，可通过
2	2092	0.624399	:	�或者其他接口��取\n [[:]]param offset: (optional) ...
3	2092	0.606773	:	:param id: ��单的ID\n [[:]]param limit: (opti...
4	2092	0.601314	:	����、����等\n [[:]]param offset: (optional) ...
5	3960	0.208631	regex	.TOKENIZERRULES_)\n :type[[regex]]ps: Tup...
6	3960	0.183832	xml	__)\n return super(AbstractAnnotationLa...
7	3960	0.180840	xsd	formats_dir+"dcoi-dsc.[[xsd]]").readlines()))...
8	3960	0.174080	DEBUG	Config``\n """\n log_level = '[[DEBUG]]'...
9	3960	0.136869	\n	\n e.addtoindex(norecurse)[...
10	4089	0.767533	username	credentials.split(':', 1)\n result[[[u...
11	4089	0.756074	id	id:\n self.doc.index[self.[[id]]] =...
12	4089	0.753835	id	in sorted(((classid, classitem) for class[[i...
13	4089	0.742677	id	id:\n self.doc.index[self.[[id]]] =...
14	4089	0.731781	id	by class ID"""\n return [classid for ...
15	3462	0.314193	to	in\n GraphQL.\n This function is simila...
16	3462	0.290129	00	'''\n ... 1\n ... 00:00...
17	3462	0.257576	d	(edit this to scale the distance between nodes...
18	3462	0.244709	://	XML/1998/namespace", 'xlink':"http[[://]]www.w...
19	3462	0.205800	00	... 00:00:00,000 --> 00:[[00]]:05,...
20	1090	0.620171	fra	milliseconds (int, float or other numeric cla...
21	1090	0.415439	])	['IGNORE_REQUEST_PATTERNS']\n [[])]]def...
22	1090	0.411837	prom	def connection_from_promised_list(data_[[prom]...
23	1090	0.408332	])	['IGNORE_SQL_PATTERNS']\n [[])]]def _du...
24	1090	0.384369	fra	and/or negative times. Non-negative times wit...

Next steps: Generate code with qualitative_context_df New interactive sheet

```
eval_groups = {
    "code": [
        "def add_numbers(a, b):\n    return a + b\n",
        "class User:\n    def __init__(self, name):\n        self.name = name\n",
```

```
        "import numpy as np\nx = np.array([1, 2, 3])\nprint(x.mean())\n",
    ],
    "medical": [
        "The patient was discharged after stable vital signs and normal oxygen saturation.",
        "The treatment plan included antibiotics, hydration, and follow-up monitoring.",
        "The physician reviewed the lab report and updated the medication dosage.",
    ],
    "legal": [
        "This agreement shall remain effective until terminated by either party.",
        "The parties agree to resolve disputes through binding arbitration.",
        "The contractor shall not disclose confidential information without consent.",
    ],
    "general_english": [
        "Both players chose not to participate in the final tournament.",
        "The city has a long history and many cultural monuments.",
        "The quick brown fox jumps over the lazy dog.",
    ],
}
```

```
@torch.no_grad()
def collect_group_acts(model, texts, hook_name, max_length=128):
    model.eval()

    all_acts = []
    all_tokens = []

    for text in texts:
        tokens = model.to_tokens(
            text,
            truncate=True,
            prepend_bos=True
        ).to(device)

        tokens = tokens[:, :max_length]

        logits, cache = model.run_with_cache(tokens)
        acts = cache[hook_name].reshape(-1, cache[hook_name].shape[-1])
        token_strs = model.to_str_tokens(tokens[0])

        acts = acts[1:]
        token_strs = token_strs[1:]

        all_acts.append(acts.detach().cpu().float())
        all_tokens.extend(token_strs)

    return torch.cat(all_acts, dim=0), all_tokens


@torch.no_grad()
def sae_group_stats(sae_model, acts):
    features = get_sae_features(sae_model, acts)
    return {
        "mean_l0": (features > 0).float().sum(dim=-1).mean().item(),
        "mean_activation": features.mean().item(),
        "active_feature_count": ((features > 0).float().mean(dim=0) > 0.01).sum().item(),
    }
```

```
import torch

@torch.no_grad()
def get_sae_features(sae_model, acts):
    """
    Returns SAE hidden feature activations for a batch of model activations.
    Works for common SAE structures.
    """

    device = next(sae_model.parameters()).device
    acts = acts.to(device)

]
    if hasattr(sae_model, "encode"):
        return sae_model.encode(acts)


    out = sae_model(acts)

    if isinstance(out, tuple):

        return out[1]


```

```
    if isinstance(out, dict):
        for key in ["features", "hidden", "acts", "z", "code"]:
            if key in out:
                return out[key]


    return out
```

```
@torch.no_grad()
def sae_group_stats(sae_model, acts):
    features = get_sae_features(sae_model, acts)

    return {
        "mean_l0": (features > 0).float().sum(dim=-1).mean().item(),
        "mean_l1": features.abs().sum(dim=-1).mean().item(),
        "mean_activation": features.mean().item(),
        "max_activation": features.max().item(),
        "dead_features": (features.sum(dim=0) == 0).sum().item(),
        "num_features": features.shape[-1],
    }
```

```
pre_stats_group = sae_group_stats(pre_code_sae, pre_acts_group)
ft_stats_group = sae_group_stats(ft_sae, ft_acts_group)

print("Pretrained SAE stats:", pre_stats_group)
print("Fine-tuned SAE stats:", ft_stats_group)
```

```
Pretrained SAE stats: {'mean_l0': 600.51611328125, 'mean_l1': 254.70230102539062, 'mean_activation': 0.06218317896127701, 'm
Fine-tuned SAE stats: {'mean_l0': 650.04833984375, 'mean_l1': 247.46603393554688, 'mean_activation': 0.060416512191295624, '
```


```
pre_stats_group = sae_group_stats(pre_code_sae, pre_acts_group)
ft_stats_group = sae_group_stats(ft_sae, ft_acts_group)

print("Pretrained SAE stats:", pre_stats_group)
print("Fine-tuned SAE stats:", ft_stats_group)
```

```
Pretrained SAE stats: {'mean_l0': 600.51611328125, 'mean_l1': 254.70230102539062, 'mean_activation': 0.06218317896127701, 'm
Fine-tuned SAE stats: {'mean_l0': 650.04833984375, 'mean_l1': 247.46603393554688, 'mean_activation': 0.060416512191295624, '
```


```
boring_tokens = {
    "\n", ".", ",", " ", "        ", "    ", ")", "(", ":", ";", "-", "_"
}

def is_boring_feature(top_tokens, threshold=0.6):
    tokens = [row["token"] for row in top_tokens]
```

Could not connect to the reCAPTCHA service. Please check your internet connection and reload to get a reCAPTCHA challenge.